

Software Architecture

Exercise – Architecture Significance

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

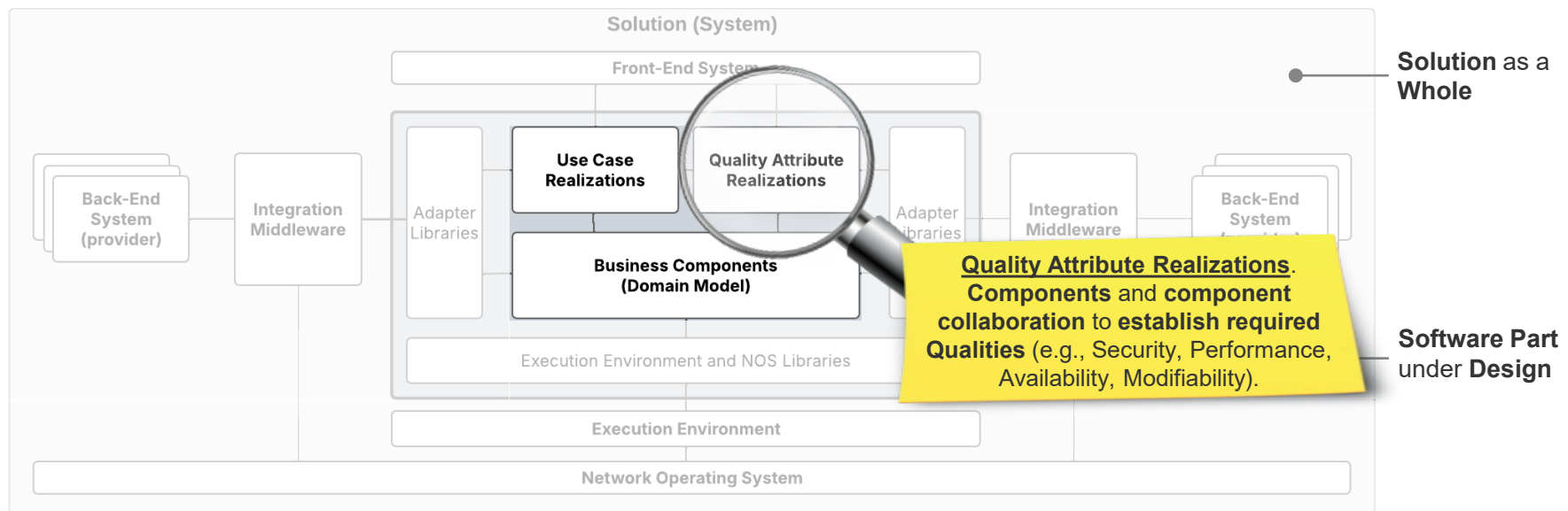
URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Exercise Opening

Architecture Context

In today's exercise, we will pursue **Quality Attribute Driven Design**, with a special focus on addressing the non-functional requirements of a solution.



Exercise Opening

Overview

In this exercise, **you will be given requirements** for a **new system** that you are **expected to design** and develop.

For **each requirement**, you need to **consider** how **architecture-significant** it is.

A **requirement is significant** if the **software design** to which the **requirement is likely to lead** is a **design** that is **difficult to dismantle, change** or **adapt** should the **requirement change** in the **future**.

Exercise Opening

Overview

You will be organised into **six working groups** and find **requirements** for a **fictitious software system** on the **following slides** (each group receives **4 requirements** to be assessed).

Your task is to ...

1. **Briefly assess** for each **requirement** whether it will **significantly shape** the **architecture** of an appropriate system.
2. For **those requirements** that **lead to far-reaching design decisions** (i.e., architecture-significant design decisions), **sketch** the **resulting design**.
 - a. **Sketch out** the **coarse-grained components** that **play a role** in your **design** (question: **which components are needed?**).
 - b. **Select** a **component** from (a) and **suggest software architecture approaches** at the **next level of detail** for this **component** (question: **which software design approaches do I propose** for a **selected component** from (a)?).
3. **Explain** to what **extent** the **design decisions** you noted would be **difficult to adapt or reverse** if the requirements changed.

Work out your **requirement assessments** based on the **Miro-Board** shared via E-Mail.

Exercise Opening Overview

List of requirements and student groups to which they are assigned.

No.	Requirement	Working Group	Significance
1	The system must be highly available 24/7.	Group 1	
2	The system should display the company logo at the top left of each page.	Group 1	
3	Users must log in with a username and password.	Group 1	
4	All system functions must also be available via a mobile app.	Group 1	
5	The system must process personal data in accordance with the GDPR.	Group 2	
6	The data entry page should include a calendar component for date fields.	Group 2	
7	The application must be available in German and English.	Group 2	
8	All business processes must be traceable and auditable for audit purposes.	Group 2	
9	The system must be integrated into the company's existing SAP environment.	Group 3	
10	The background colour of the start page should be blue.	Group 3	
11	The app should also be usable in landscape mode.	Group 3	
12	The solution should be cloud-native (e.g. on AWS).	Group 3	
13	The system should be configurable by business users without developer support.	Group 4	
14	Users should be able to sort data records by columns.	Group 4	
15	Password policies should require at least 8 characters and one capital letter.	Group 4	
16	The system must be fully multi-client capable.	Group 4	

Exercise Opening

Overview

List of requirements and student groups to which they are assigned.

No.	Requirement	Working Group	Significance
17	The system must meet real-time requirements (e.g. responses within 50 ms).	Group 5	
18	The system should automatically log out after 15 minutes of inactivity.	Group 5	
19	Input fields should perform validations (e.g. email format).	Group 5	
20	The system must be expandable by third parties (e.g. partner companies) via publicly documented interfaces.	Group 5	
21	The system must support machine learning (e.g. by training and using models based on usage data).	Group 6	
22	The application should offer a print function for invoices.	Group 6	
23	The system should allow users to reset their passwords themselves.	Group 6	
24	The system must support rule-based decision logic that can be adapted by specialists at runtime.	Group 6	

Exercise Opening

Requirements Assessment

Reference solution for
yet another requirement



example

Requirement: The **system** must be **capable** of **offline operation** (e.g., for mobile use without network coverage).

Reference solution for
yet another requirement



example

Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable of offline operation** (e.g., for mobile use without network coverage).

Architecture Significance: Yes.

Reference solution for
yet another requirement



example

Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable** of **offline operation** (e.g., for mobile use without network coverage).

Architecture Significance: Yes.

Why:

This **requirement necessitates** profound **changes** to **data storage**, **state management**, and **synchronization logic**.

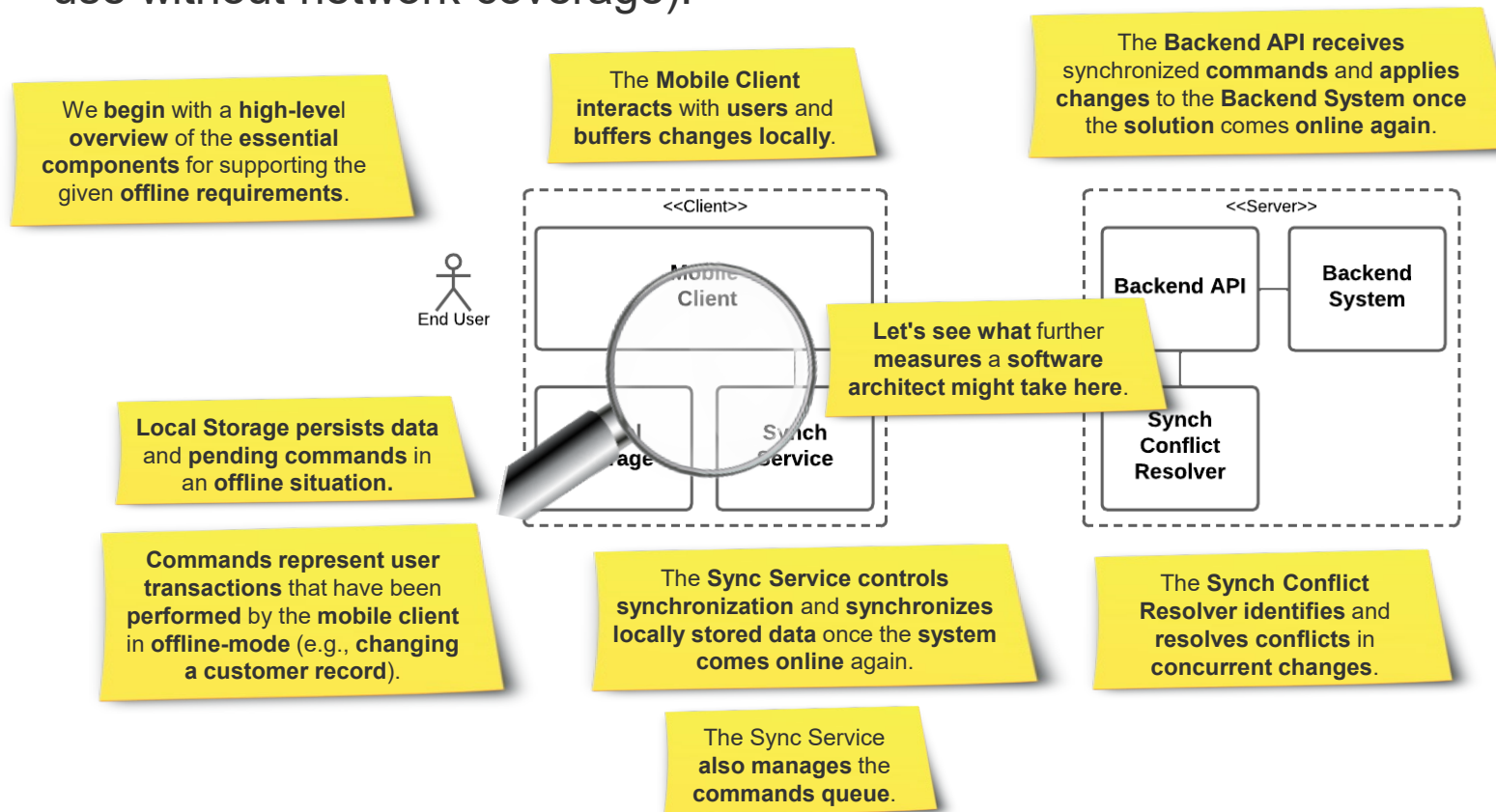
It also **leads** to **design considerations** and **decisions** regarding **conflict handling**, **versioning**, and **reconciliation strategies**.



Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable of offline operation** (e.g., for mobile use without network coverage).

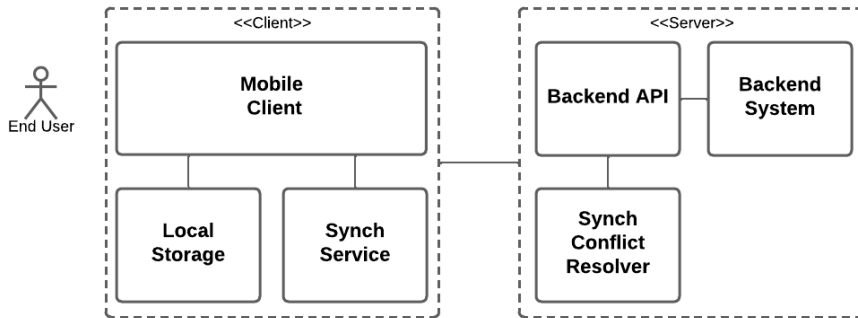




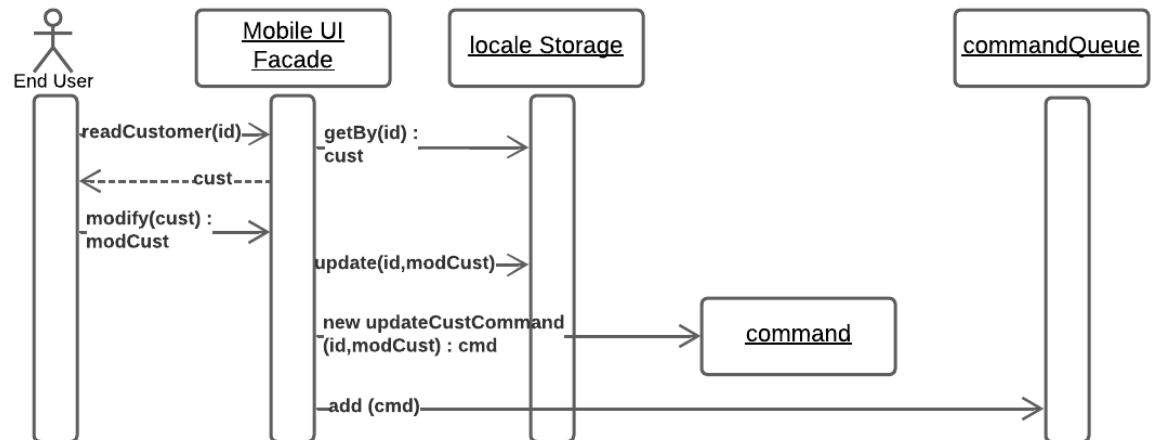
Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable of offline operation** (e.g., for mobile use without network coverage).



The **sequence diagram** shows the **cooperation** between key **components** when **modifying** a **customer record** in **offline mode**.

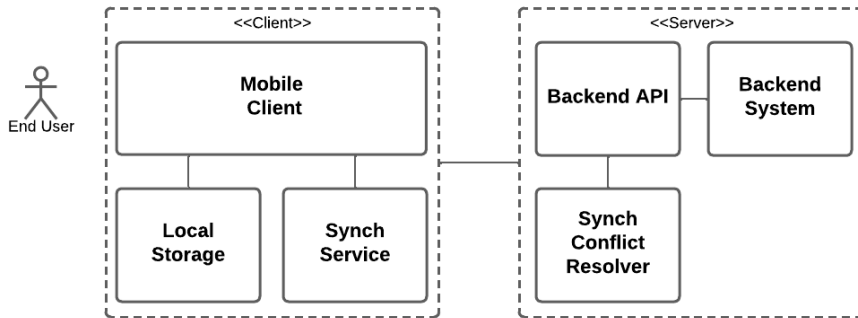




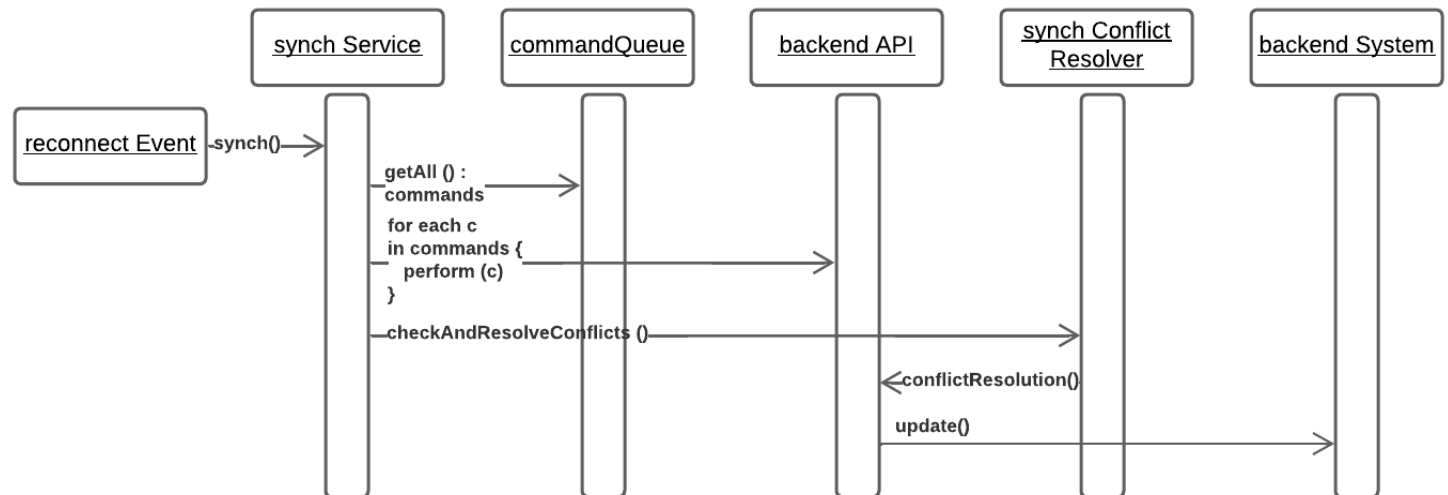
Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable of offline operation** (e.g., for mobile use without network coverage).



As soon as the **mobile app comes back online**, a **corresponding event triggers synchronization with the backend**.

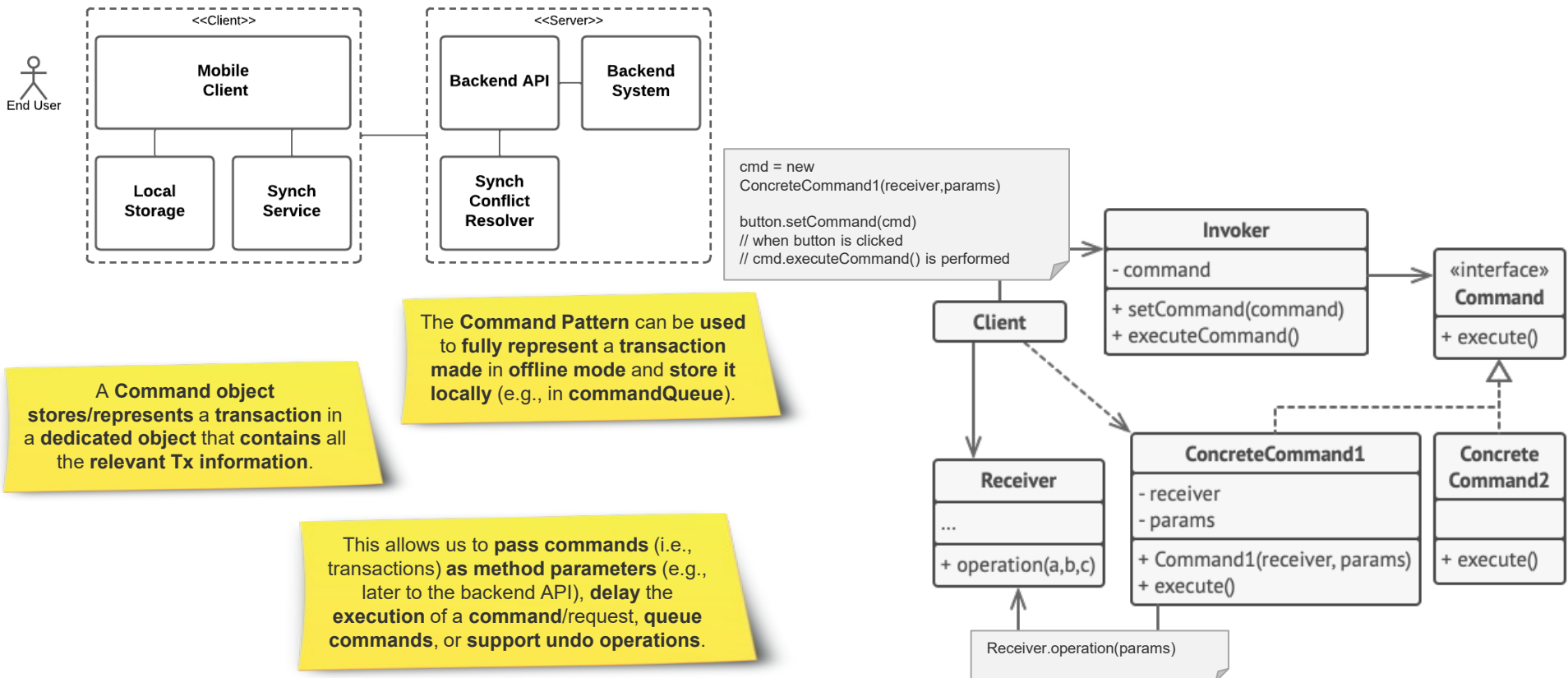




Exercise Opening

Requirements Assessment

Requirement: The **system** must be **capable of offline operation** (e.g., for mobile use without network coverage).





example

Exercise Opening

Requirements Assessment

Requirement 1: The system must be highly available 24/7.

Architecture Significance: Yes.

Resulting Architecture Decisions:

This requirement usually leads to the introduction of redundancy (clusters, load balancers, failover systems), distributed system components and, if necessary, separate monitoring infrastructure.

Why difficult to change Architecture Decisions later:

Once implemented, a high-availability setup affects deployment, testability, operating models and monitoring.

It is expensive to change if, for example, requirements change towards 5x9 availability (instead of 24/7) or lower operating requirements.



example

Exercise Opening

Requirements Assessment

Requirement 2: The system should display the company logo at the top left of every page.

Architecture Significance: No.

Resulting Architecture Decisions:

This concerns pure UI presentation. It does not lead to any specific structural or behavioural architecture decisions.



example

Exercise Opening

Requirements Assessment

Requirement 3: A user must log in with a user name and password.

Architecture Significance: No.

Resulting Architecture Decisions:

Standard requirement that can be implemented with existing authentication modules.
Architecture remains largely unaffected.



example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.

Architecture Significance: Yes.

Resulting Architecture Decisions:

This necessitates an API layer with appropriate backend decoupling and, if necessary, specific security measures (tokens, OAuth2, device binding).

Why difficult to change Architecture Decisions later:

The design of an API layer with abstracted endpoints and mobile authentication flows affects the data model, state processing and logging functions system-wide.

Downgrading to a browser-centric front end is complex.

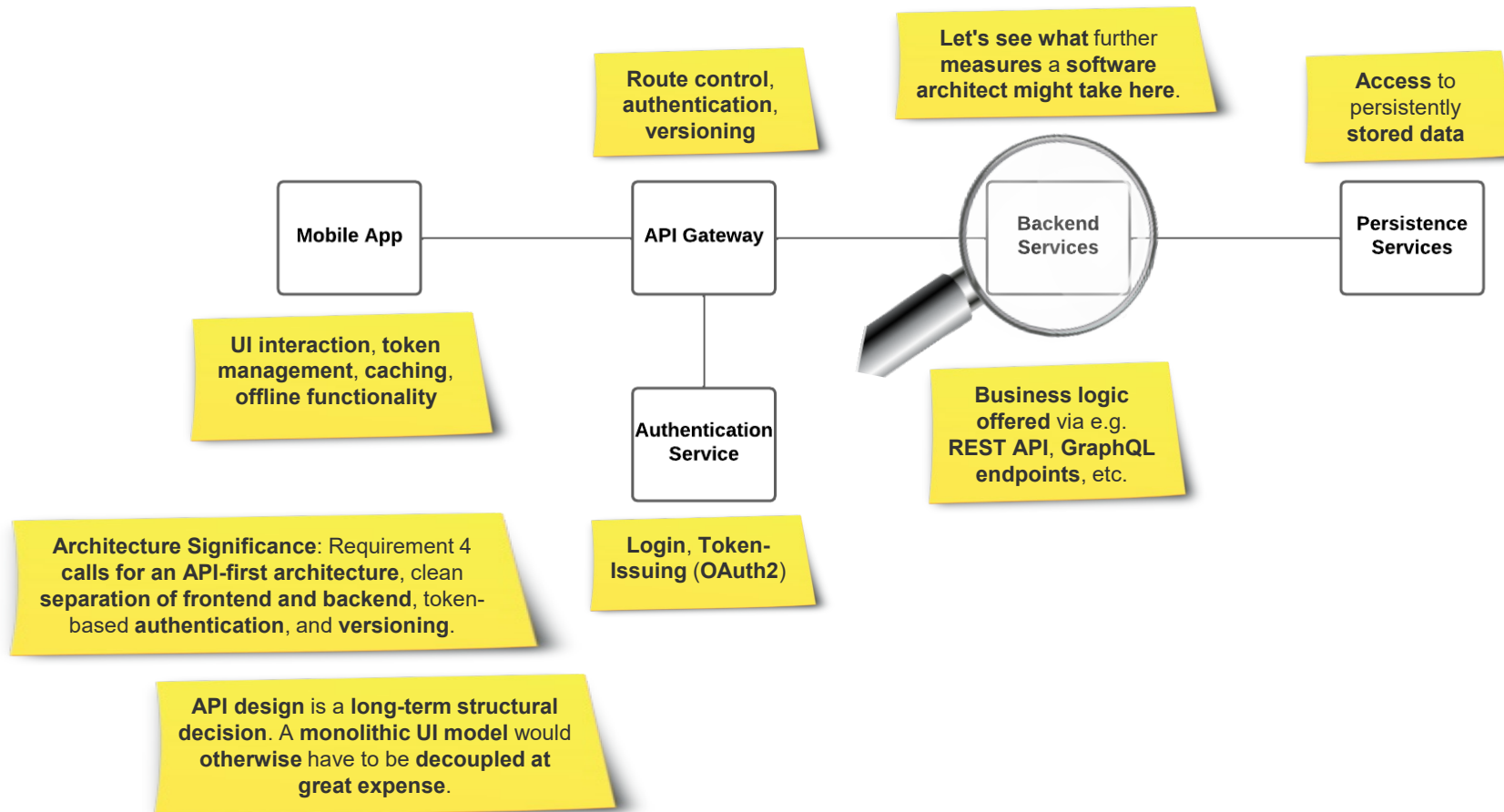


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



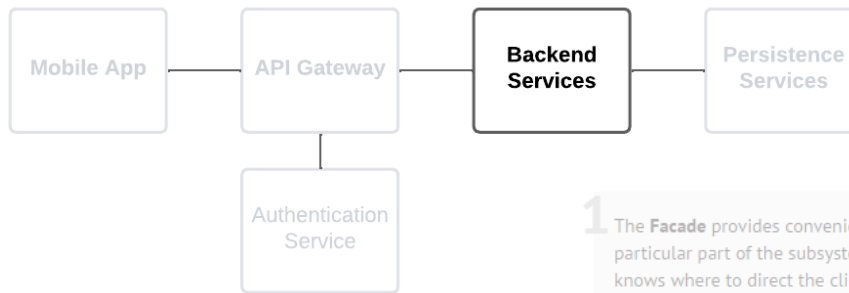


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



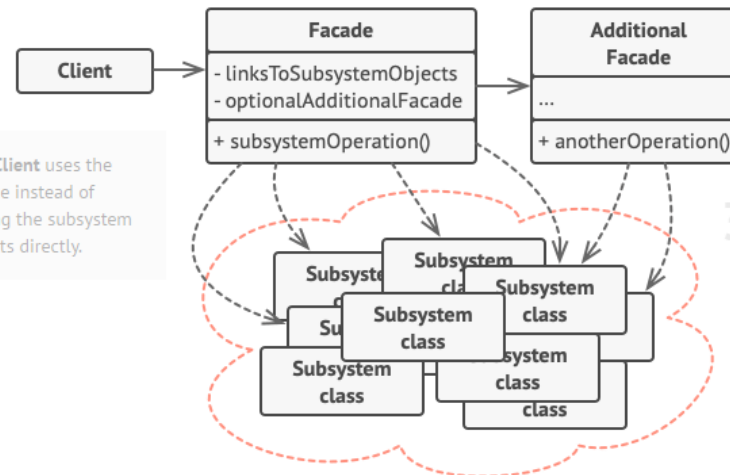
The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize Façades. Services (APIs) can be **encapsulated** by **façade classes** that **provide mobile-specific use cases** (e.g., “load overview,” “save photo”).

1 The **Facade** provides convenient access to a particular part of the subsystem's functionality. It knows where to direct the client's request and how to operate all the moving parts.

2 An **Additional Facade** class can be created to prevent polluting a single facade with unrelated features that might make it yet another complex structure. Additional facades can be used by both clients and other facades.

4 The **Client** uses the facade instead of calling the subsystem objects directly.



3 The **Complex Subsystem** consists of dozens of various objects. To make them all do something meaningful, you have to dive deep into the subsystem's implementation details, such as initializing objects in the correct order and supplying them with data in the proper format.

Subsystem classes aren't aware of the facade's existence. They operate within the system and work with each other directly.

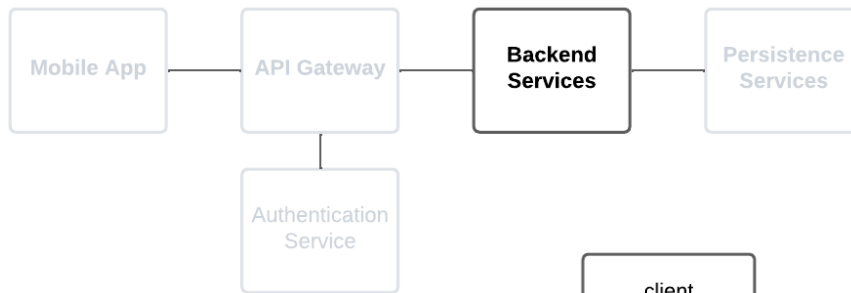


example

Exercise Opening

Requirements Assessment

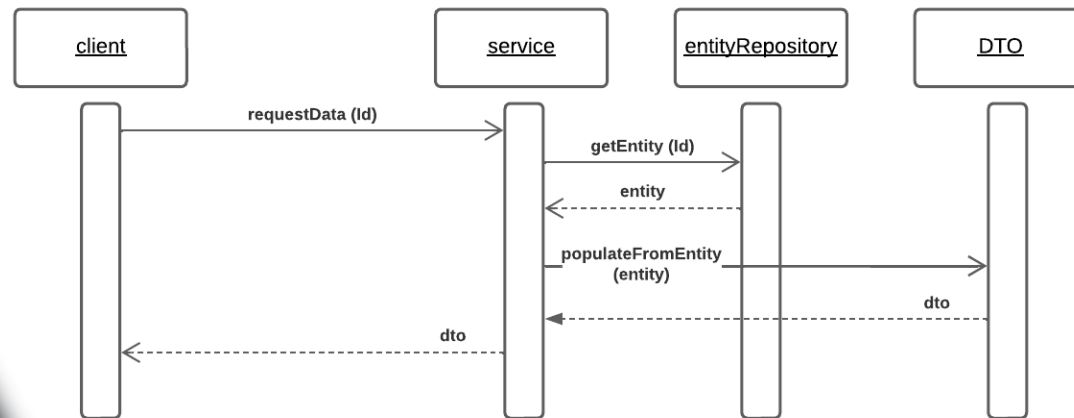
Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize **DTOs** (Data Transfer Objects) whenever **responses** must **deliver clearly structured, mobile-optimized data** (e.g., flat structures, no cyclic references).

Software Architects use the **DTO-Pattern**, when **network traffic** needs to be **optimized** by **reducing the number of calls**, especially in a client-server architecture.



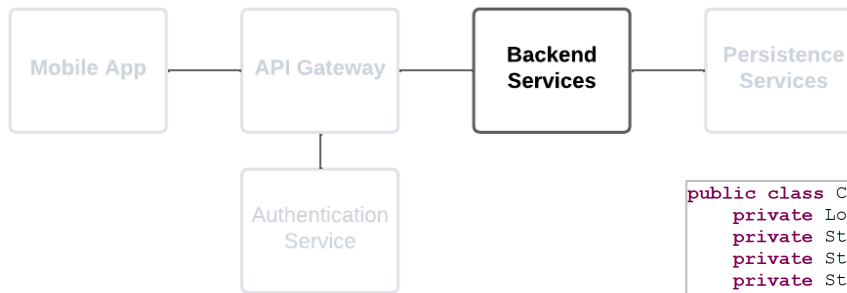


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize **DTOs** (Data Transfer Objects) whenever **responses** must **deliver clearly structured, mobile-optimized data** (e.g., flat structures, no cyclic references).

Software Architects use the **DTO-Pattern**, when **network traffic** needs to be **optimized** by **reducing the number of calls**, especially in a client-server architecture.

Classes of business entities (like **Customer** in this example) contain **business logic** and are **relatively heavyweight**.

```

public class Customer {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private int loyaltyPoints;

    public Customer(Long id, String firstName, String lastName, String email,
        int loyaltyPoints) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.loyaltyPoints = loyaltyPoints;
    }

    // Getter und Setter
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }

    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public int getLoyaltyPoints() { return loyaltyPoints; }
    public void setLoyaltyPoints(int loyaltyPoints) {
        this.loyaltyPoints = loyaltyPoints;
    }
}
  
```

```

// Business Logic: complete name
public String getFullName() {
    return firstName + " " + lastName;
}

// Business Logic: Check of customer is premium customer
// (z. B. ab 1000 Treuepunkten)
public boolean isPremiumCustomer() {
    return loyaltyPoints >= 1000;
}

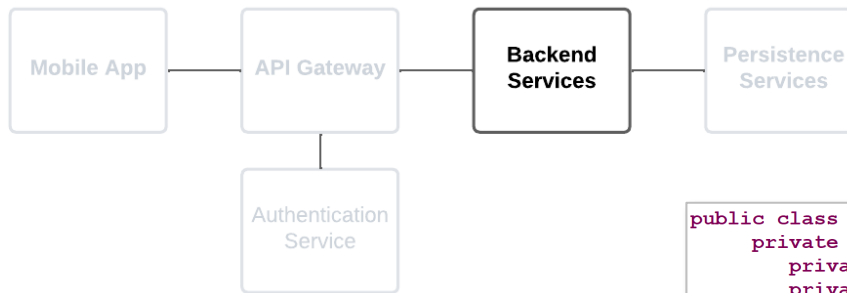
// Business Logic: additional Loyalty Points
public void addLoyaltyPoints(int additionalPoints) {
    if (additionalPoints > 0) {
        loyaltyPoints += additionalPoints;
    }
}
  
```



Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize **DTOs** (Data Transfer Objects) whenever **responses** must **deliver clearly structured, mobile-optimized data** (e.g., flat structures, no cyclic references).

Software Architects use the **DTO-Pattern**, when **network traffic** needs to be **optimized** by **reducing the number of calls**, especially in a client-server architecture.

A **DTO class** (e.g., **CustomerDTO**) is a **representation** of a **corresponding business entity** (e.g., **Customer**) reduced to **essential data points**.

```
public class CustomerDTO {
    private Long id;
    private String fullName;
    private String email;
    private int loyaltyPoints;

    public CustomerDTO(Long id, String fullName, String email, int loyaltyPoints) {
        this.id = id;
        this.fullName = fullName;
        this.email = email;
        this.loyaltyPoints = loyaltyPoints;
    }

    // Getter und Setter
    public int getLoyaltyPoints() { return loyaltyPoints; }
    public void setLoyaltyPoints(int loyaltyPoints) {
        this.loyaltyPoints = loyaltyPoints;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

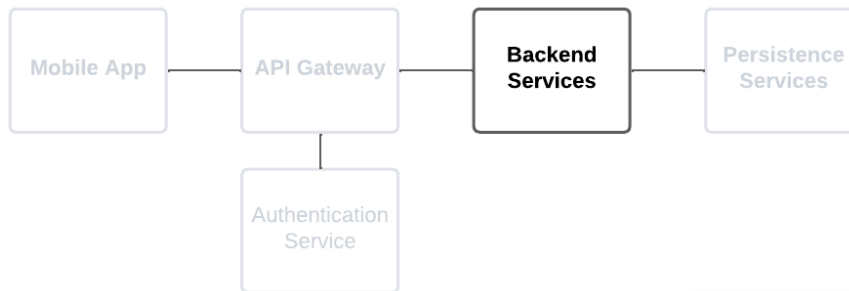


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize **DTOs** (Data Transfer Objects) whenever **responses** must **deliver clearly structured, mobile-optimized data** (e.g., flat structures, no cyclic references).

Software Architects use the **DTO-Pattern**, when **network traffic** needs to be **optimized** by **reducing the number of calls**, especially in a client-server architecture.

For the transition between **DTO** and corresponding business entity, there are typically **mapper classes** (**CustomerMapper**) that support mapping in both directions.

```
public class CustomerMapper {

    public static CustomerDTO toDTO(Customer customer) {
        return new CustomerDTO(
            customer.getId(),
            customer.getFullName(),
            customer.getEmail(),
            customer.getLoyaltyPoints()
        );
    }

    public static Customer fromDTO(CustomerDTO dto) {
        // Since the DTO only contains the fullName, this is simply split
        String[] parts = dto.getFullName().split(" ", 2);
        String firstName = parts.length > 0 ? parts[0] : "";
        String lastName = parts.length > 1 ? parts[1] : "";
        return new Customer(dto.getId(), firstName, lastName, dto.getEmail(),
            dto.getLoyaltyPoints());
    }
}
```

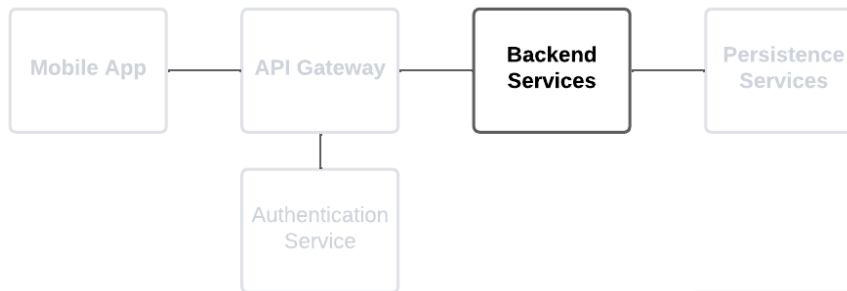



example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize **DTOs** (Data Transfer Objects) whenever **responses** must **deliver clearly structured, mobile-optimized data** (e.g., flat structures, no cyclic references).

Software Architects use the **DTO-Pattern**, when **network traffic** needs to be **optimized** by **reducing the number of calls**, especially in a client-server architecture.

```
public class Client {

    public static void main(String[] args) {
        Customer customer = new Customer(1L,
            "Ingo", "Arnold",
            "ingo.arnold@unibas.ch", 1);

        // Domain → DTO
        CustomerDTO dto = CustomerMapper.toDTO(customer);
        System.out.println(dto.getFullName()); // Output: Ingo Arnold

        // DTO → Domain
        Customer reconstructed = CustomerMapper.fromDTO(dto);
        System.out.println(reconstructed.getFirstName()); // Output: Ingo
    }
}
```

Finally, the **main method** above shows how **everything** is **typically** put together.

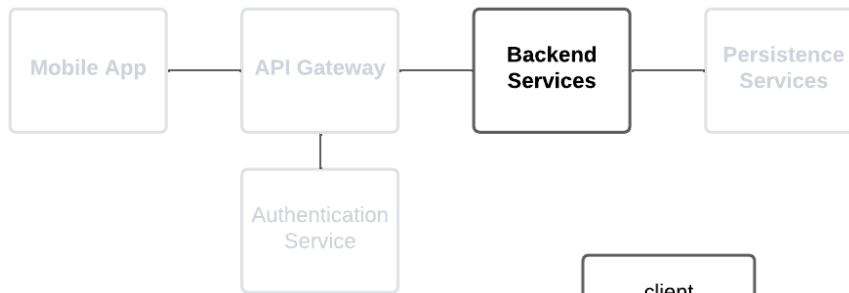


example

Exercise Opening

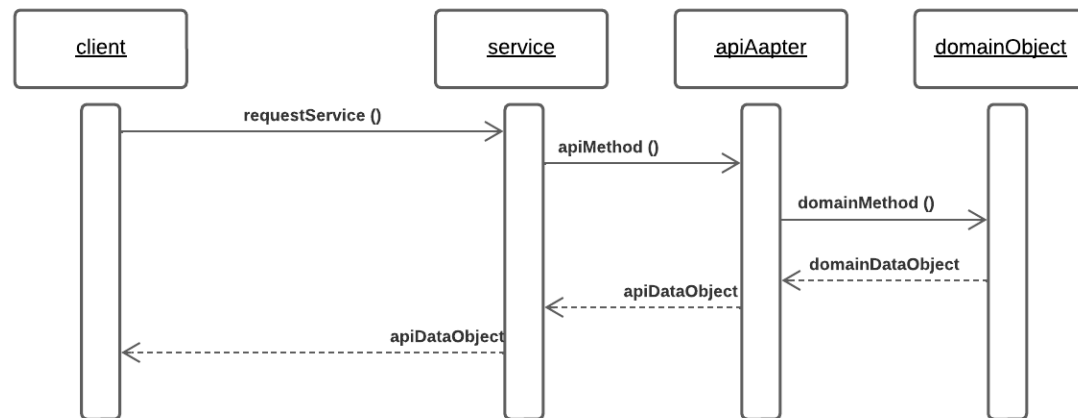
Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize the adapter pattern to **moderate between domain-specific objects** and a client-facing **API layer**.
An **Adapter** separates **external representation** from **internal domain logic**.



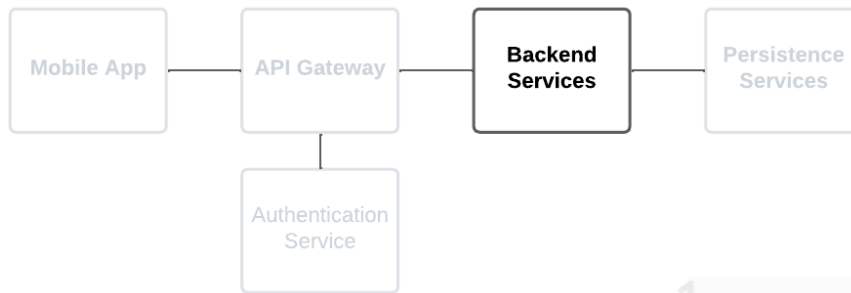


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize the adapter pattern to **moderate between domain-specific objects** and a client-facing **API layer**.
An **Adapter** separates **external representation** from **internal domain logic**.

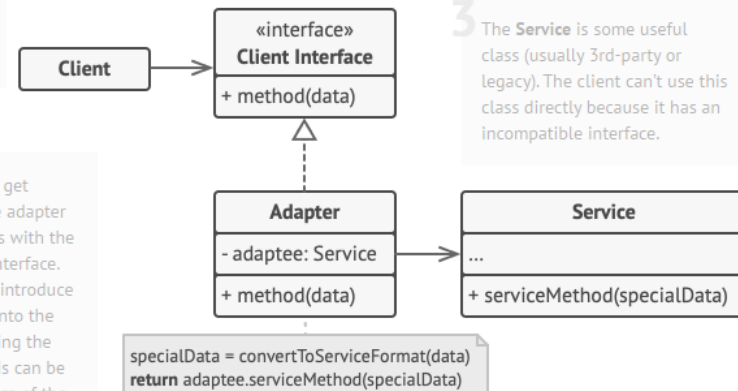
- Two types of adapters** are distinguished:
- Adapter with **delegation (object adapter)**
 - Adapter with **inheritance (class adapter)**

1 The **Client** is a class that contains the existing business logic of the program.

2 The **Client Interface** describes a protocol that other classes must follow to be able to collaborate with the client code.

3 The **Service** is some useful class (usually 3rd-party or legacy). The client can't use this class directly because it has an incompatible interface.

5 The client code doesn't get coupled to the concrete adapter class as long as it works with the adapter via the client interface. Thanks to this, you can introduce new types of adapters into the program without breaking the existing client code. This can be useful when the interface of the service class gets changed or replaced: you can just create a new adapter class without changing the client code.



4 The **Adapter** is a class that's able to work with both the client and the service: it implements the client interface, while wrapping the service object. The adapter receives calls from the client via the adapter interface and translates them into calls to the wrapped service object in a format it can understand.

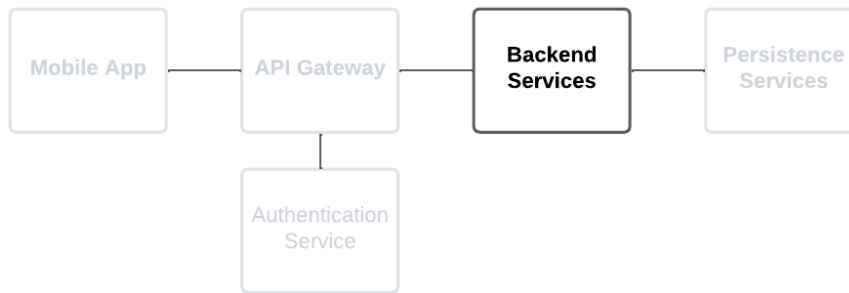


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.

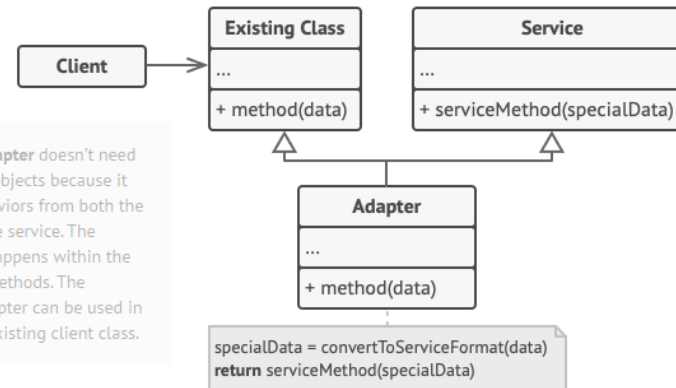


The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Software Architects utilize the adapter pattern to **moderate between domain-specific objects** and a client-facing **API layer**.
An **Adapter** separates **external representation** from **internal domain logic**.

- Two types of adapters** are distinguished:
- Adapter with **delegation (object adapter)**
 - Adapter with **inheritance (class adapter)**

1 The **Class Adapter** doesn't need to wrap any objects because it inherits behaviors from both the client and the service. The adaptation happens within the overridden methods. The resulting adapter can be used in place of an existing client class.



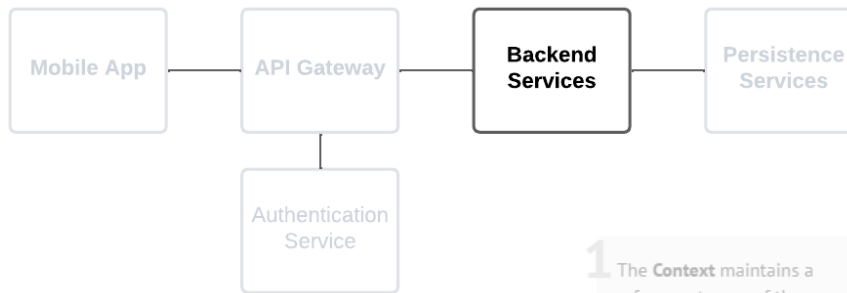


example

Exercise Opening

Requirements Assessment

Requirement 4: All system functions must also be available via a mobile app.



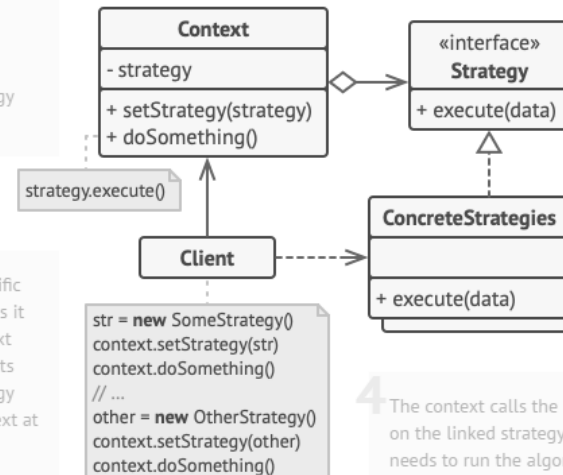
The clear **objective** here is to **structure backend services** in such a way that **mobile clients** can **access** them in a **stable, secure, and efficient** manner.

Finally, **Software Architects** may achieve versioning by adapting the strategy pattern.

Strategy enables parallel API versions (v1, v2, etc.) within the same code via interchangeable strategy implementations.

1 The **Context** maintains a reference to one of the concrete strategies and communicates with this object only via the strategy interface.

5 The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter which lets clients replace the strategy associated with the context at runtime.



2 The **Strategy** interface is common to all concrete strategies. It declares a method the context uses to execute a strategy.

3 **Concrete Strategies** implement different variations of an algorithm the context uses.

4 The context calls the execution method on the linked strategy object each time it needs to run the algorithm. The context doesn't know what type of strategy it works with or how the algorithm is executed.



example

Exercise Opening

Requirements Assessment

Requirement 5: The system must process personal data in accordance with the GDPR

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to the introduction of data protection mechanisms (audit trails, pseudonymisation, right to be forgotten, access control at attribute level).

Why difficult to change Architecture Decisions later:

Data protection-friendly design must be conceived at an early stage.

If the system was not designed from the outset to separate sensitive and non-sensitive data, separating them later is very time-consuming and prone to errors.



example

Exercise Opening

Requirements Assessment

Requirement 6: The data entry page should contain a calendar component for date fields.

Architecture Significance: No.

Resulting Architecture Decisions:

UI component – no significant impact on architecture or system structure.



example

Exercise Opening

Requirements Assessment

Requirement 7: The application must be available in German and English.

Architecture Significance: No.

Resulting Architecture Decisions:

Internationalisation (i18n) is often handled via libraries; rarely affects the system architecture in the strict sense.



example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to the introduction of event stores, audit logs, versioned data storage, immutable objects (immutability), and separation of live and historical data.

Why difficult to change Architecture Decisions later:

A system that was not built with an audit target image usually does not have suitable historization.

Subsequent introduction of complete traceability (e.g. at field level) requires changes to almost all persistence, logging and transaction mechanisms.

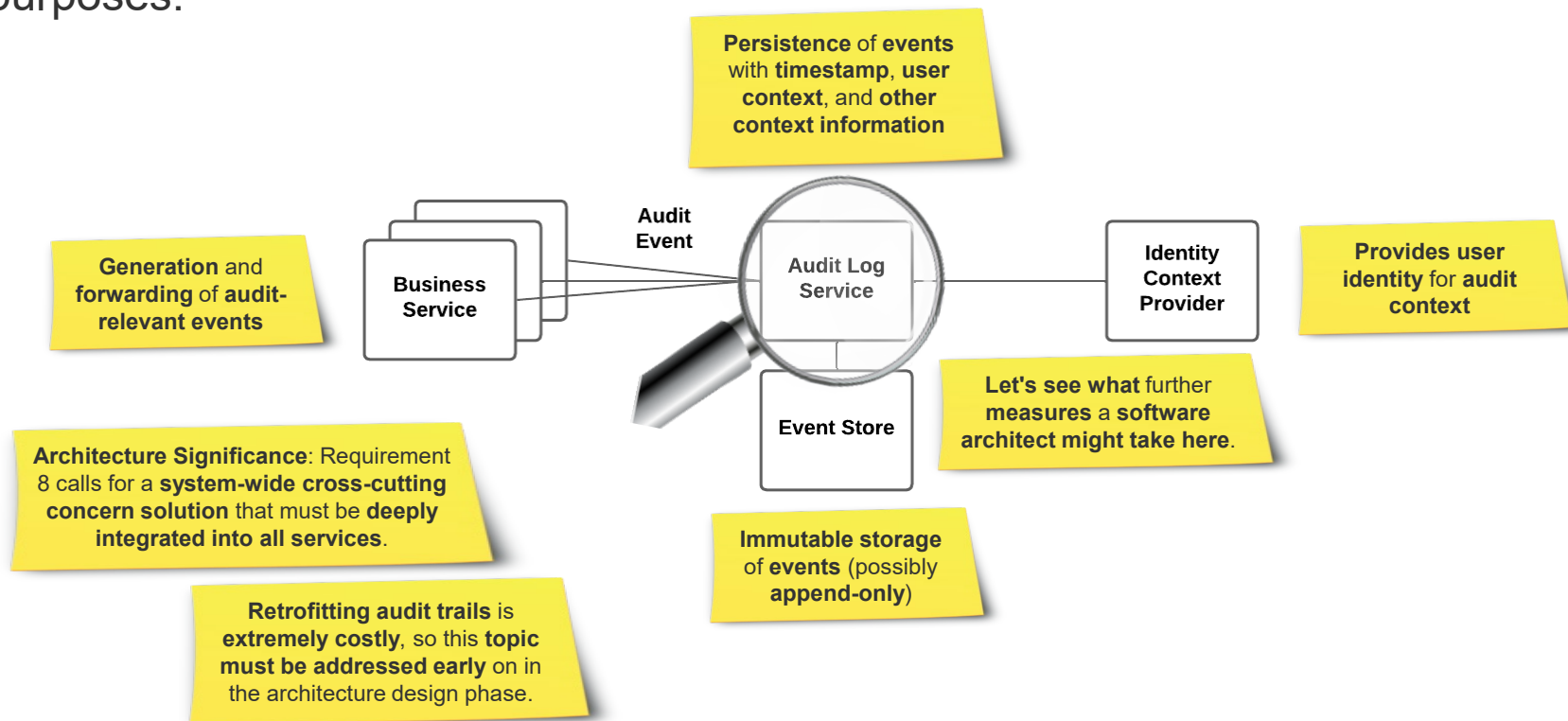


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



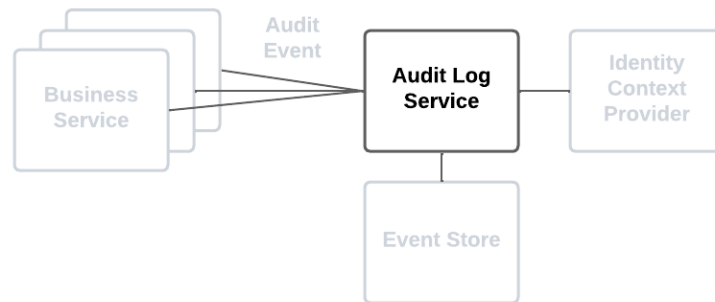


example

Exercise Opening

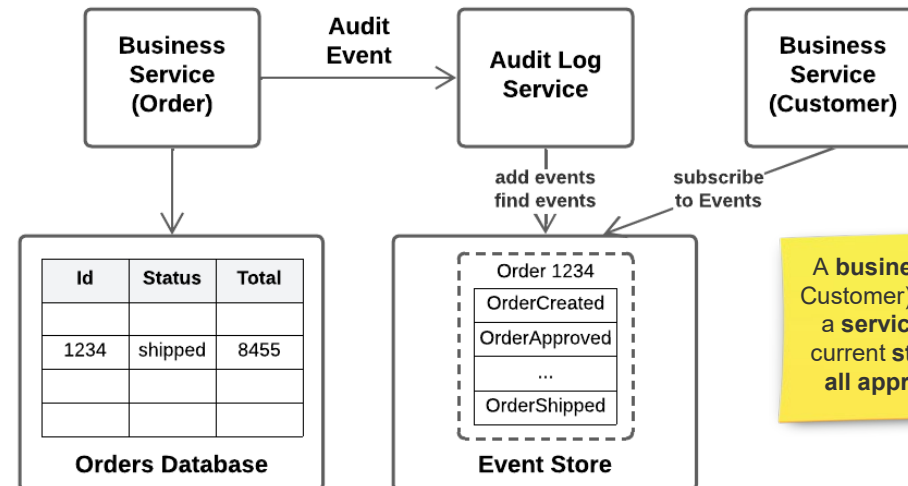
Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

Adapting the Event Sourcing Pattern (Architecture Pattern) your design persists the state of a business service (e.g., Order or Customer service) as a sequence of state-changing events.



A business service (e.g., Customer) may reconstruct a service's (e.g., Order) current state by replaying all appropriate events.

Whenever the state of a business service changes, a new event is appended to the list of events.

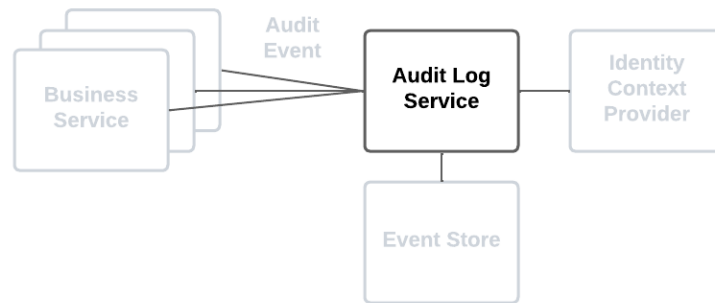


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

Furthermore, the **Observer Pattern** could be adapted for the **Audit Log Service** so that the **Audit Log Service** registers as an **observer** with **Business Service(s)** and is hence **automatically informed of state changes in business services**.

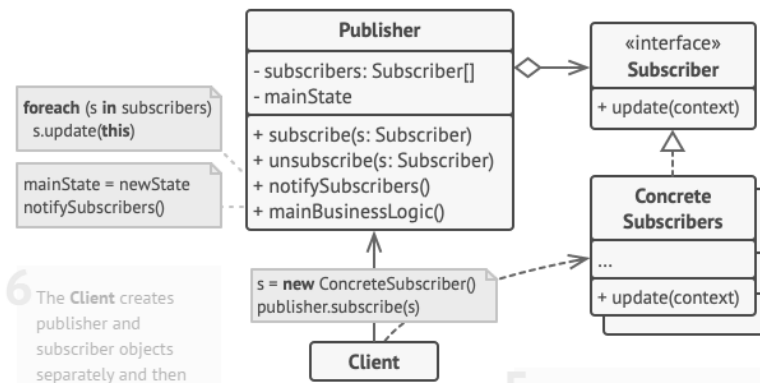
However, this approach would require the introduction of deliberate log calls in every business service, which would mix business and technical (logging) logic.

1 The **Publisher** issues events of interest to other objects. These events occur when the publisher changes its state or executes some behaviors. Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.

2 When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.

3 The **Subscriber** interface declares the notification interface. In most cases, it consists of a single `update` method. The method may have several parameters that let the publisher pass some event details along with the update.

4 **Concrete Subscribers** perform some actions in response to notifications issued by the publisher. All of these classes must implement the same interface so the publisher isn't coupled to concrete classes.



6 The **Client** creates publisher and subscriber objects separately and then registers subscribers for publisher updates.

5 Usually, subscribers need some contextual information to handle the update correctly. For this reason, publishers often pass some context data as arguments of the notification method. The publisher can pass itself as an argument, letting subscriber fetch any required data directly.

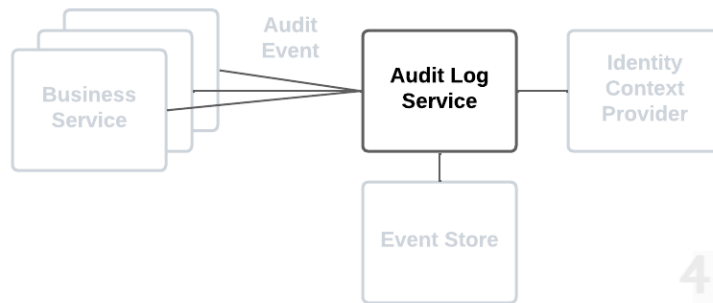


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

A pattern that would help here is **Proxy**.

Proxy is a structural design pattern that can be used to provide a substitute or placeholder for another object.

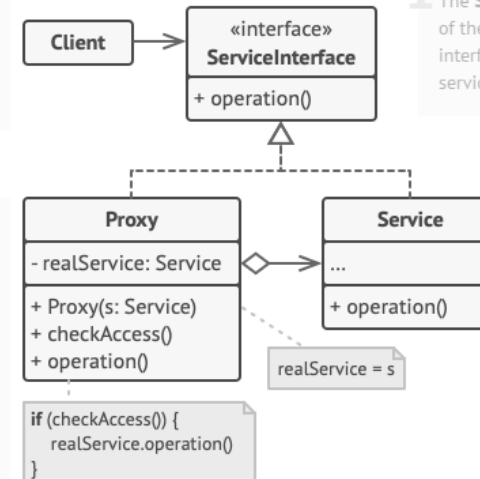
A proxy intercepts access to the original object, allowing something to be executed before or after the request to the original object (for example, sending a log event to the audit log service).

By wrapping a business service in a business service proxy and placing the logging in the proxy, we relieve the business service from having notify registered observers (via LogEvents) itself.

4 The **Client** should work with both services and proxies via the same interface. This way you can pass a proxy into any code that expects a service object.

3 The **Proxy** class has a reference field that points to a service object. After the proxy finishes its processing (e.g., lazy initialization, logging, access control, caching, etc.), it passes the request to the service object.

Usually, proxies manage the full lifecycle of their service objects.



1 The **Service Interface** declares the interface of the Service. The proxy must follow this interface to be able to disguise itself as a service object.

2 The **Service** is a class that provides some useful business logic.

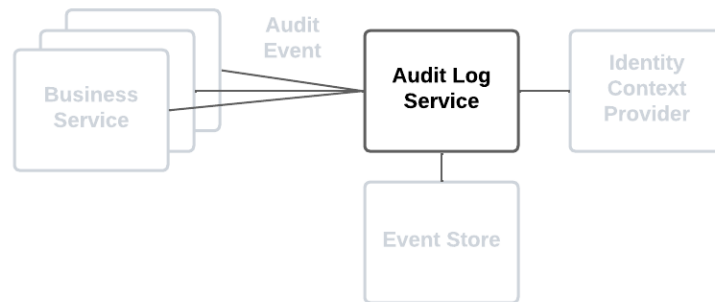


example

Exercise Opening

Requirements Assessment

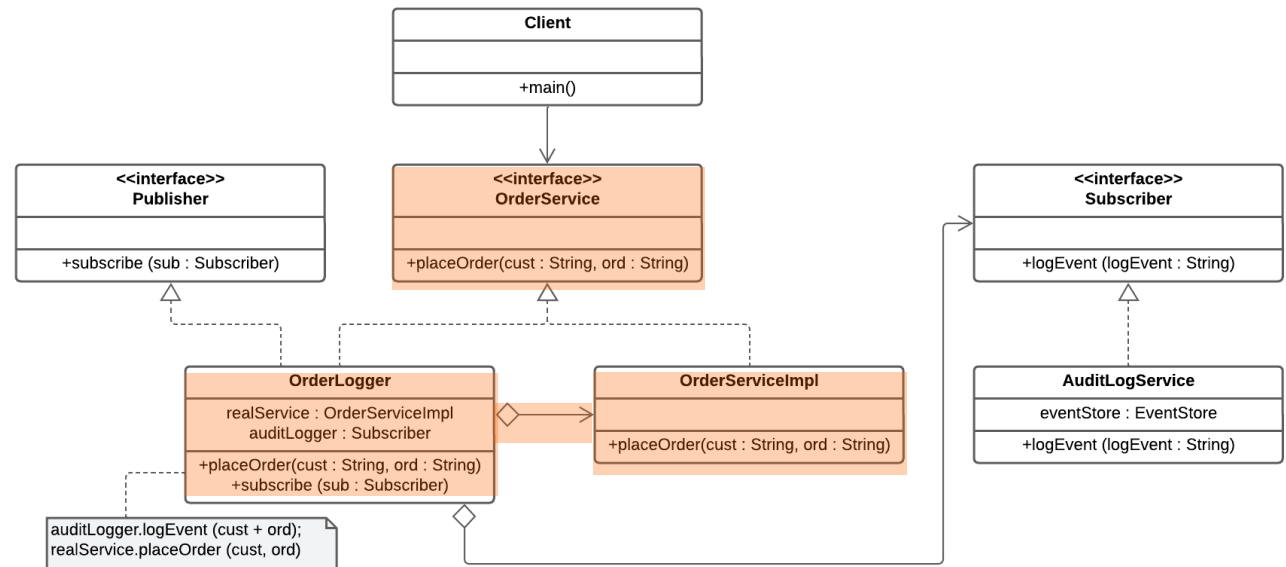
Requirement 8: All business processes must be traceable and auditable for audit purposes.



The **goal** is to **collect audit data centrally**, **store it securely** and **efficiently**, and **make it reconstructable** if necessary.

Let's **combine the two patterns (observer and proxy)** in concrete terms and **see what design they lead to** in the form of a **class diagram**.

These classes adapt the **Proxy pattern**.



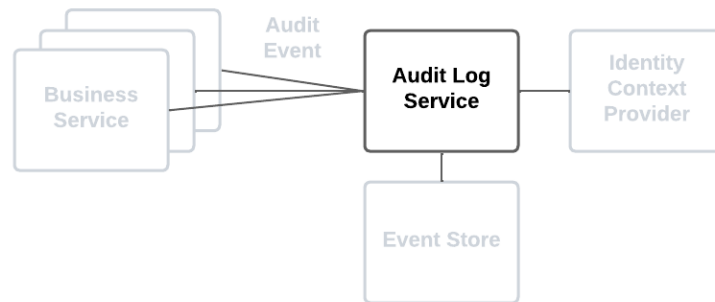


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.

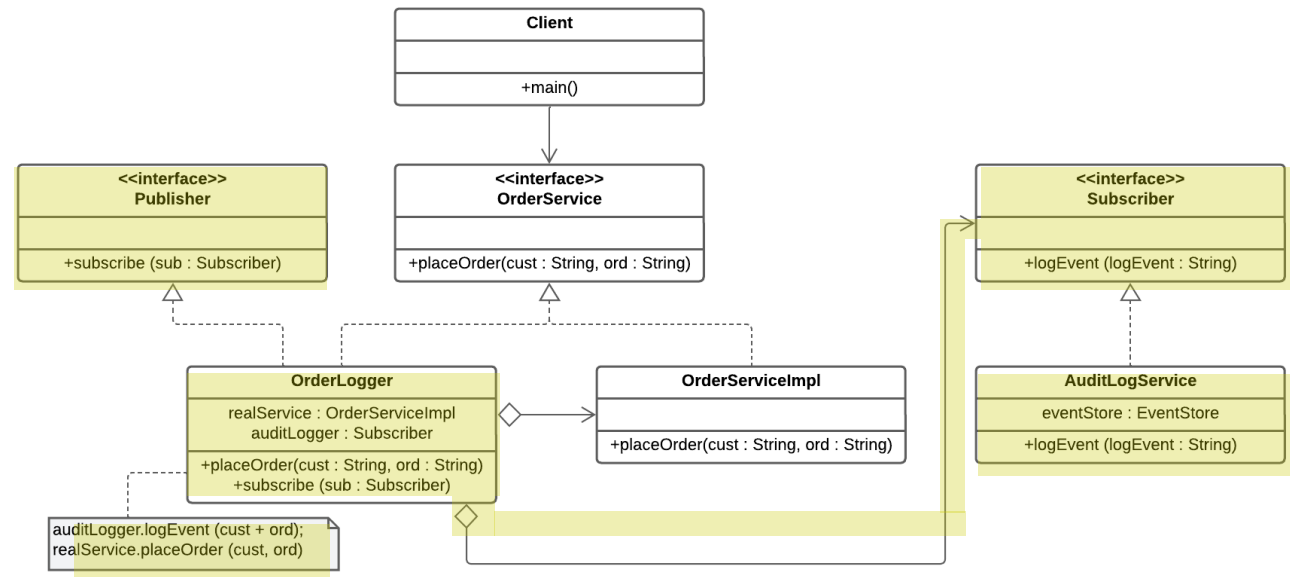


The **goal** is to **collect audit data centrally**, **store it securely** and **efficiently**, and **make it reconstructable** if necessary.

Let's **combine the two patterns (observer and proxy)** in concrete terms and **see what design they lead to** in the form of a **class diagram**.

These classes adapt the **Proxy pattern**.

And these **classes adapt the Observer pattern**.



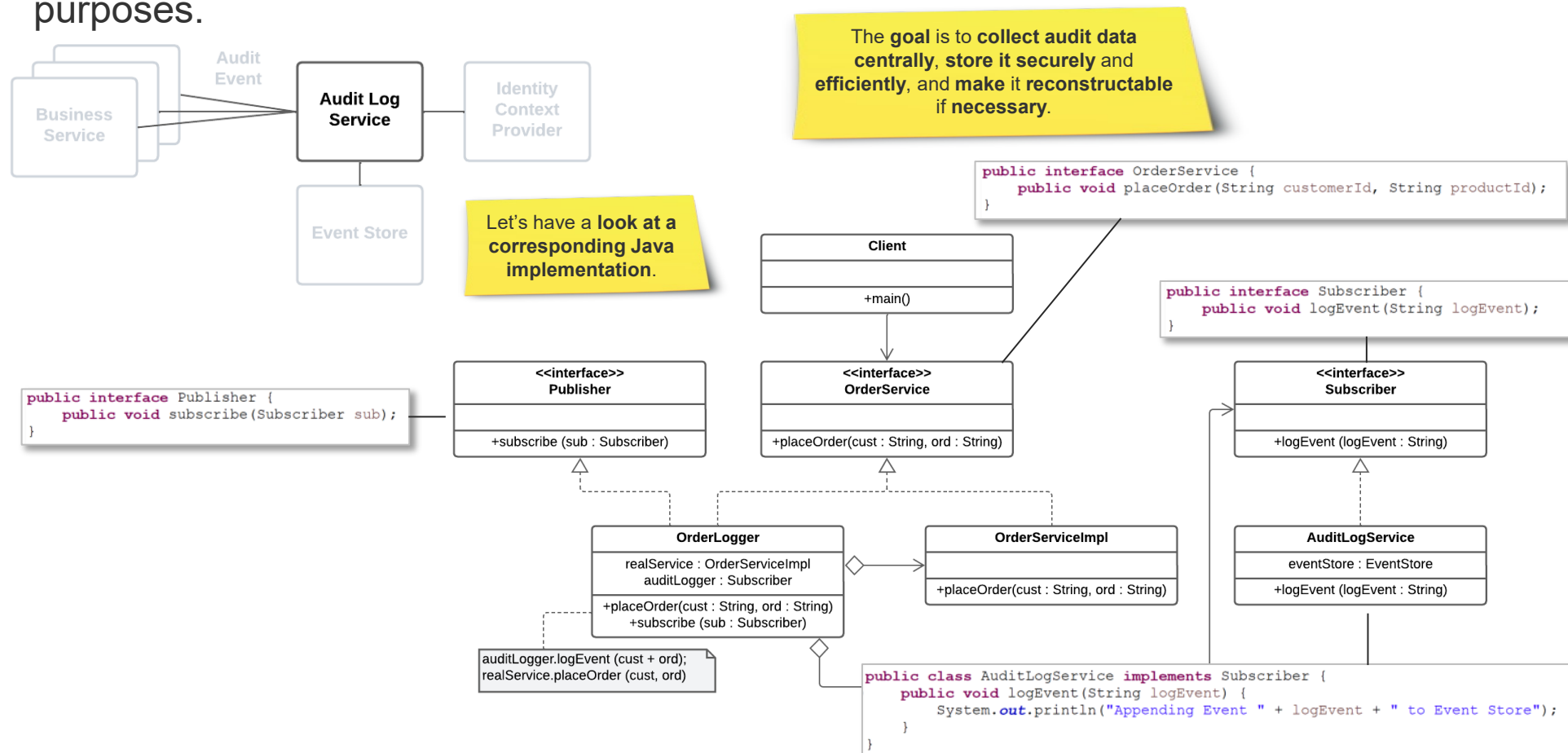


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



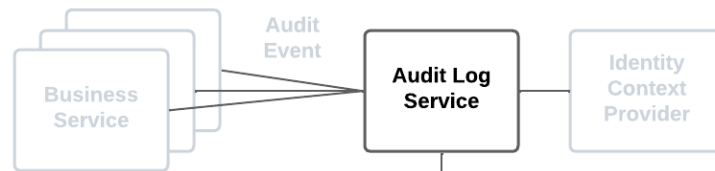


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

```

public class OrderLogger implements Publisher, OrderService {
    private OrderService orderService = null;
    private Subscriber logService = null;

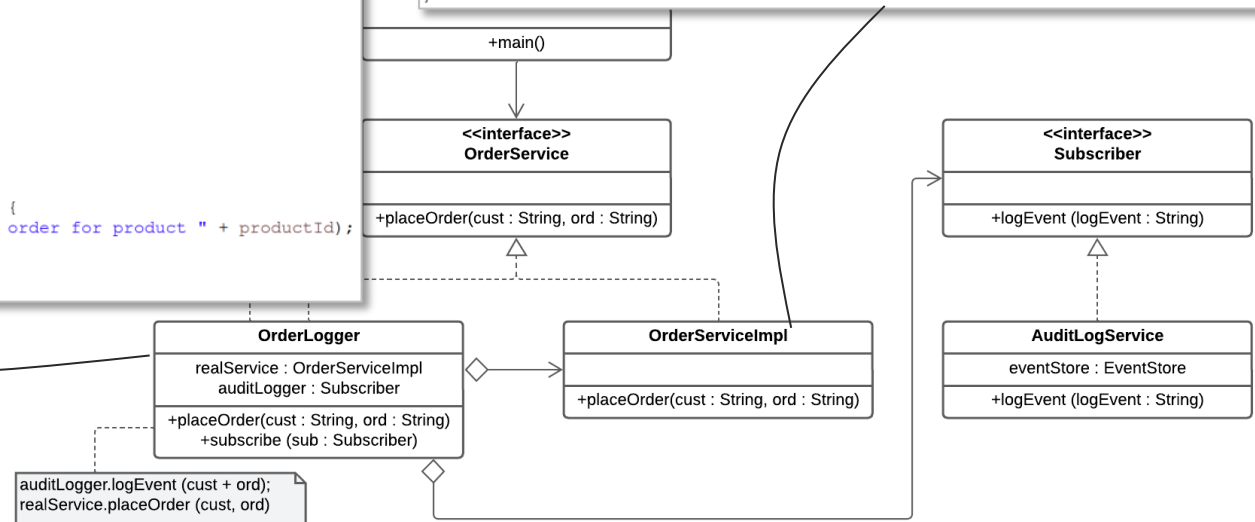
    public OrderLogger(OrderService orderService) {
        this.orderService = orderService;
    }

    public void subscribe(Subscriber logService) {
        this.logService = logService;
    }

    public void placeOrder(String customerId, String productId) {
        logService.logEvent("Customer " + customerId + " places order for product " + productId);
        orderService.placeOrder(customerId, productId);
    }
}
  
```

```

public class OrderServiceImpl implements OrderService {
    public void placeOrder(String customerId, String productId) {
        System.out.println("Placing order for customer " + customerId + " in DB");
    }
}
  
```



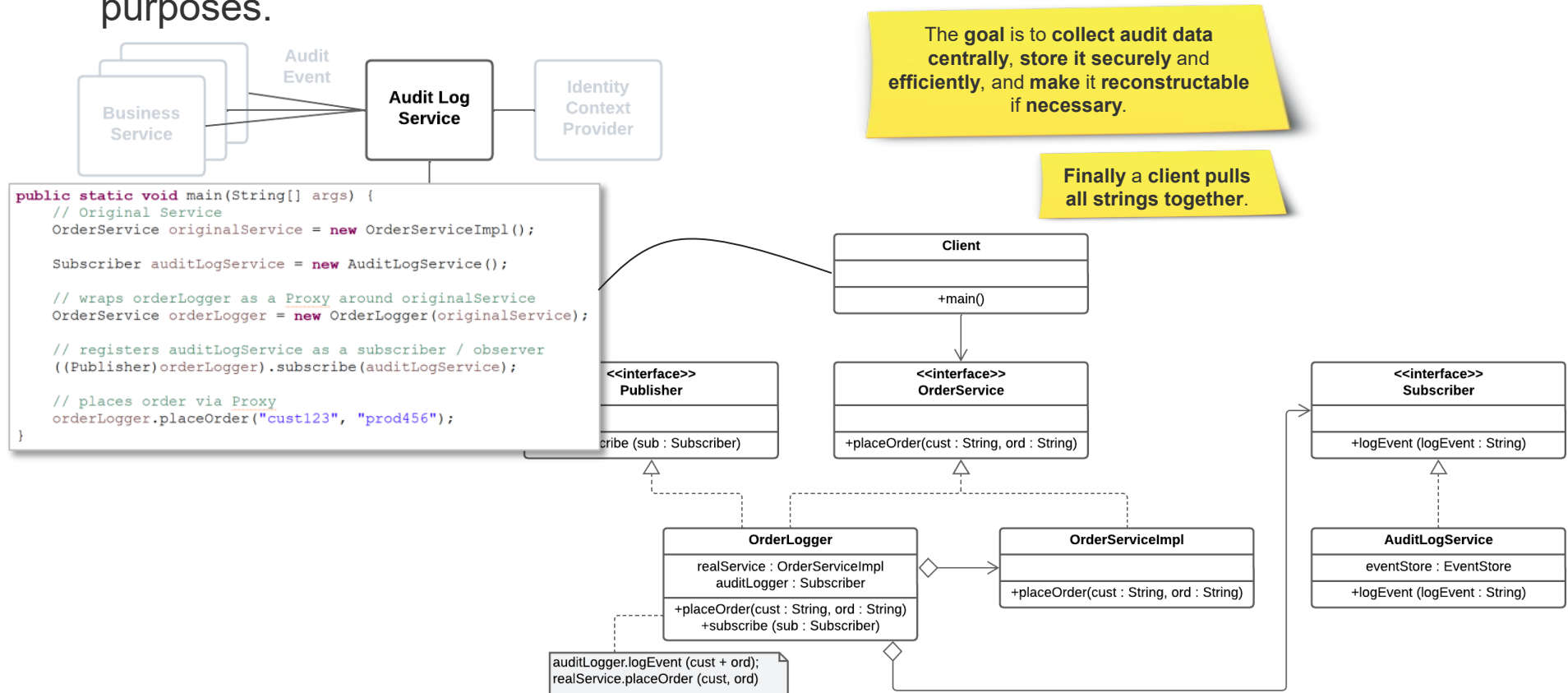


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



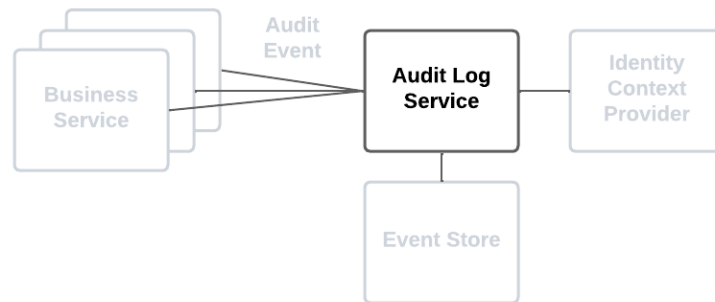


example

Exercise Opening

Requirements Assessment

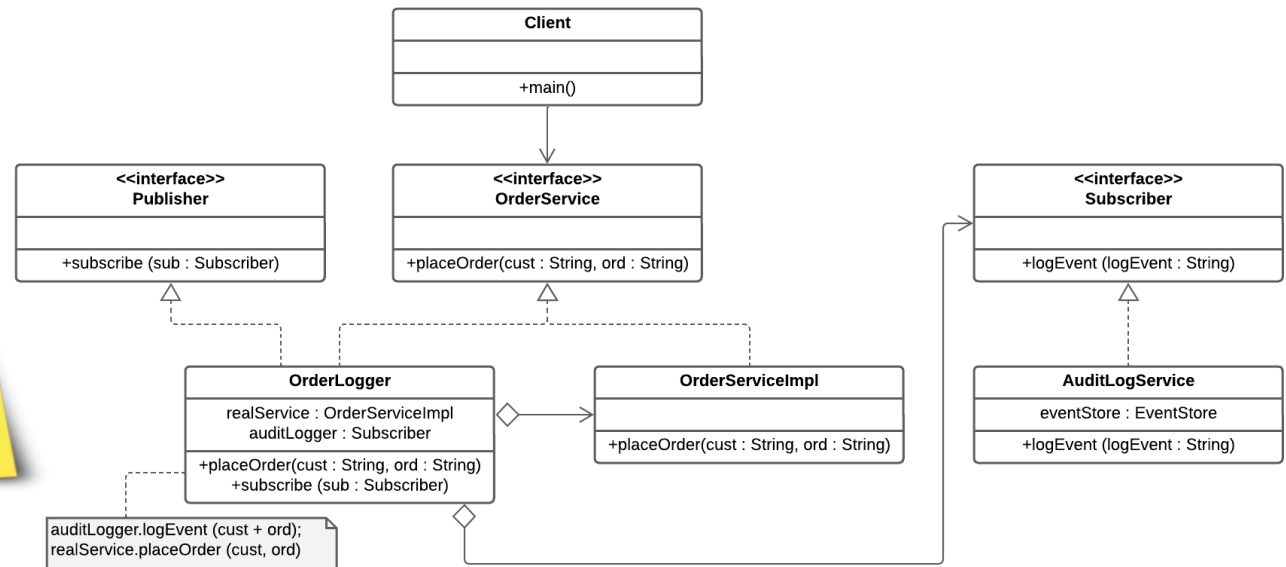
Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

In this design, the business service (OrderServiceImpl) is completely freed from knowledge of logging.

At the same time, a specific logger proxy (dedicated class) must be created for each business service.



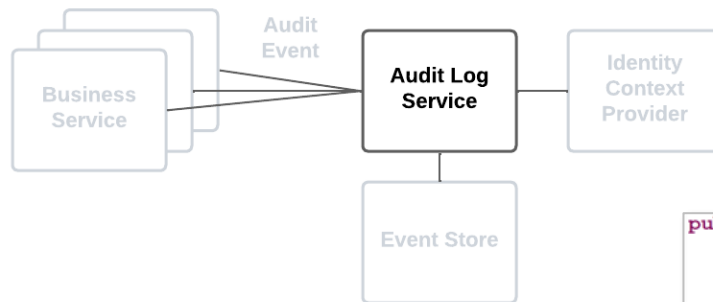


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

The design could be further improved by a dynamic proxy – a feature offered since JDK 1.3 via the `java.lang.reflect.Proxy` interface.

In an appropriately adjusted design, most interfaces and classes remain the same as before.

However, the approach for the AutoLogger proxy would change fundamentally.

A generic proxy class would now be used (i.e., one proxy class for all business services) ...

```

public class AuditLoggingHandler implements InvocationHandler {

    private final Object target;
    private final Subscriber subscriber;

    public AuditLoggingHandler(Object target, Subscriber subscriber) {
        this.target = target;
        this.subscriber = subscriber;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        // Audit Log before method execution
        String logEvent = "[AUDIT] Calling method: " + method.getName();
        if (args != null) {
            for (Object arg : args) {
                logEvent += " [ARG] " + arg;
            }
        }
        subscriber.logEvent(logEvent);

        // Execution of actual method
        Object result = method.invoke(target, args);

        return result;
    }
}
  
```

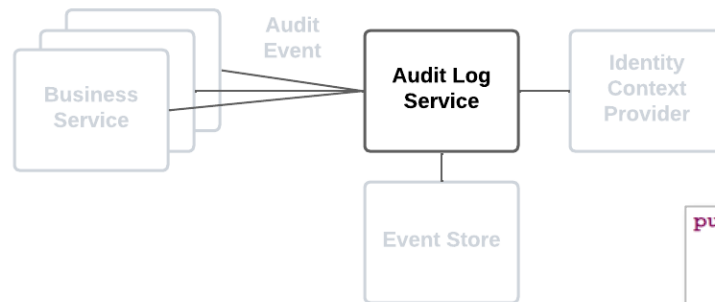


example

Exercise Opening

Requirements Assessment

Requirement 8: All business processes must be traceable and auditable for audit purposes.



The goal is to collect audit data centrally, store it securely and efficiently, and make it reconstructable if necessary.

The design could be further improved by a dynamic proxy – a feature offered since JDK 1.3 via the `java.lang.reflect.Proxy` interface.

... and utilized in a respective main method as shown here ...

```
public static void main(String[] args) {  
    // Original Service  
    OrderService originalService = new OrderServiceImpl();  
    Subscriber auditLogService = new AuditLogService();  
  
    // Dynamic Proxy  
    OrderService proxyService = (OrderService) Proxy.newProxyInstance(  
        OrderService.class.getClassLoader(),  
        new Class<?>[] { OrderService.class },  
        new AuditLoggingHandler(originalService, auditLogService)  
    );  
  
    // Execution via Proxy - automatically audited  
    proxyService.placeOrder("cust123", "prod456");  
}
```



example

Exercise Opening

Requirements Assessment

Requirement 9: The system must be integrated into the company's existing SAP environment.

Architecture Significance: Yes.

Resulting Architecture Decisions:

This leads to the selection of specific integration patterns (e.g. RFC, IDoc, BAPI, event bridges) and restrictions in the choice of platform and technology stack.

Why difficult to change Architecture Decisions later:

Integration into an ERP system affects all system layers – from the domain model to the deployment mechanism.

Decoupling at a later stage is expensive and risky.



example

Exercise Opening

Requirements Assessment

Requirement 10: The background colour of the start page should be blue.

Architecture Significance: No.

Resulting Architecture Decisions:

Pure UI configuration – does not affect architectural elements.



example

Exercise Opening

Requirements Assessment

Requirement 11: The app should also be usable in landscape mode.

Architecture Significance: No.

Resulting Architecture Decisions:

Design question at the UI level (e.g. responsive design), affects layout strategies, not the architecture.



example

Exercise Opening

Requirements Assessment

Requirement 12: The solution should be cloud-native (e.g. on AWS).

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to a microservice-based structure, containerised deployments, use of platform services (e.g. S3, Lambda) and DevOps practices.

Why difficult to change Architecture Decisions later:

A subsequent switch to a monolithic on-premise system would be virtually impossible without redesigning the entire architecture.

Switching to a different cloud platform can also be difficult (lock-in effects).

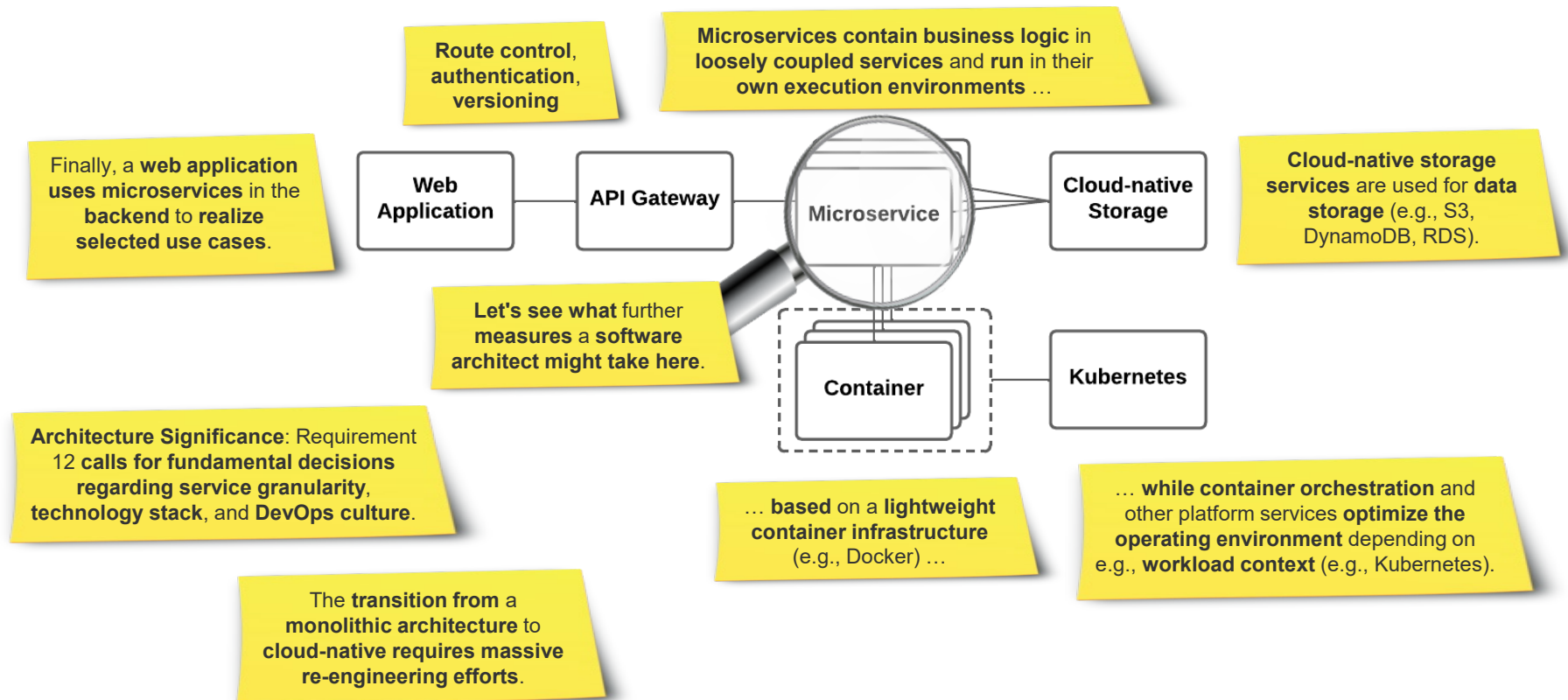


example

Exercise Opening

Requirements Assessment

Requirement 12: The solution should be cloud-native (e.g. on AWS).



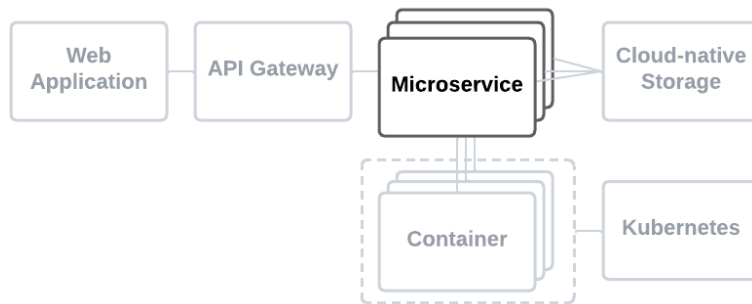


example

Exercise Opening

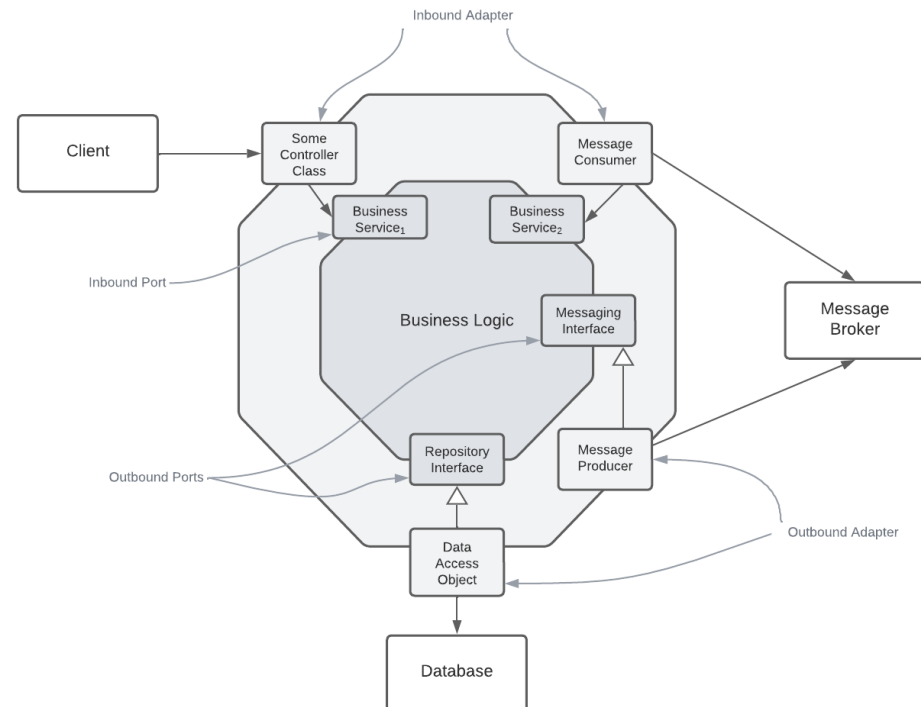
Requirements Assessment

Requirement 12: The solution should be cloud-native (e.g. on AWS).



Adapting a hexagonal architecture approach (ports & adapters) promotes strict separation between business logic (internal) and context-adaptations (external (e.g., client, API, database, message broker)).

An important goal here is to **structure Microservices** in such a way that they are **lightweight, independently deployable, and scalable**.



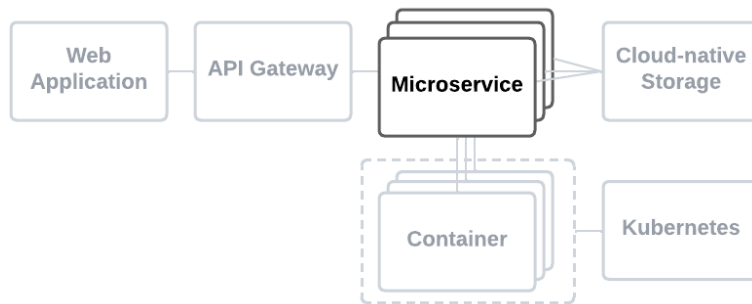


example

Exercise Opening

Requirements Assessment

Requirement 12: The solution should be cloud-native (e.g. on AWS).



An **important goal** here is to **structure Microservices** in **such a way** that they are **lightweight, independently deployable, and scalable**.

This is **illustrated** by the **many equivalent instances** of the **order service**.

This principle requires that a **service does not store** any **information between service-requests**, or that it **fully delegates** persisting any **status information** to **specific components**.

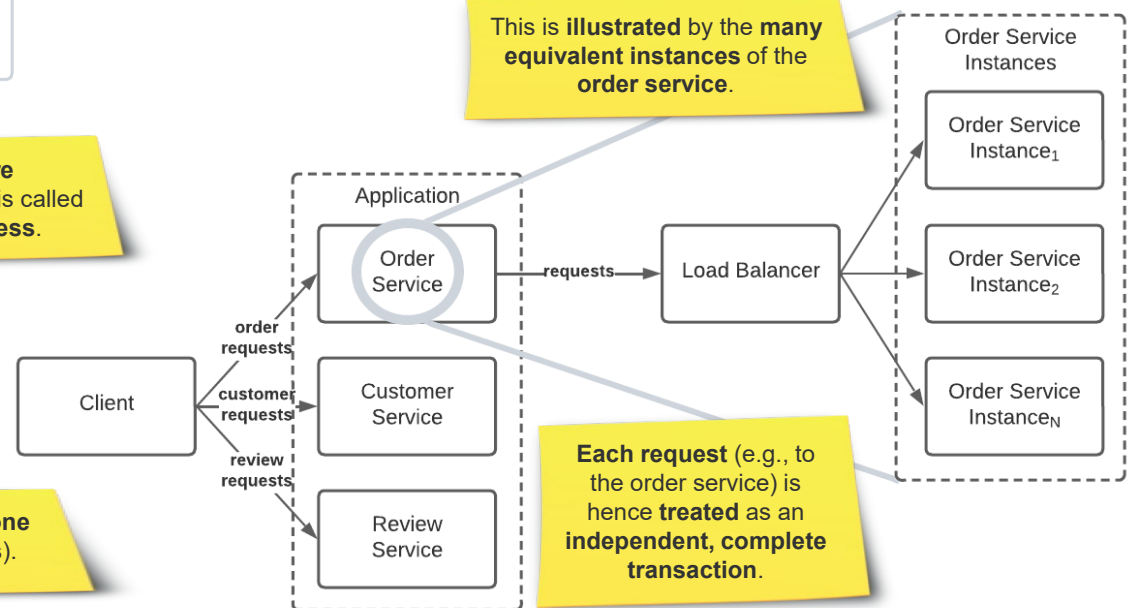
Another software architecture principle is called **service statelessness**.

... **easy scalability** (since any server can process the request)

This offers major **advantages** such as ...

high fault tolerance (since the **failure of one server** does **not affect the overall status**).

... and **improved agility and maintainability**, since the **complexity of status management** is **delegated** to **external components** (e.g., central database).



Each request (e.g., to the order service) is hence **treated** as an **independent, complete transaction**.



example

Exercise Opening

Requirements Assessment

Requirement 13: The system should be configurable by business users without developer support.

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to the introduction of a set of rules, configuration UI, script engines or rule-based evaluation logic. This means separation of static code and dynamic behaviour.

Why difficult to change Architecture Decisions later:

Such a metamodel has a profound impact on the architecture.

It requires separate test procedures, validation mechanisms and UI elements – and is difficult to dismantle because logic has been ‘moved out of the code’.



example

Exercise Opening

Requirements Assessment

Requirement 14: Users should be able to sort data records by columns.

Architecture Significance: No.

Resulting Architecture Decisions:

Typically implemented in the client component (front end); no structural architecture implications.



example

Exercise Opening

Requirements Assessment

Requirement 15: Password policies should require at least 8 characters and one capital letter.

Architecture Significance: No.

Resulting Architecture Decisions:

A security policy that can be implemented in an authentication module (or input validation on the level of UI forms); no profound impact on architecture.



example

Exercise Opening

Requirements Assessment

Requirement 16: The system must be fully multi-client capable.

Architecture Significance: Yes.

Resulting Architecture Decisions:

Multi-tenancy requires far-reaching decisions at the data level: separate databases, schema sharing or shared schemas with tenant identifiers. It also affects all access controls, caching strategies, logging and configuration models.

Why difficult to change Architecture Decisions later:

A system that has not been built to be multi-tenant can only be converted with great effort – partly because of deeply embedded assumptions about data and session isolation.

Changing the tenant strategy (e.g. from shared to isolated schema) is also complex.

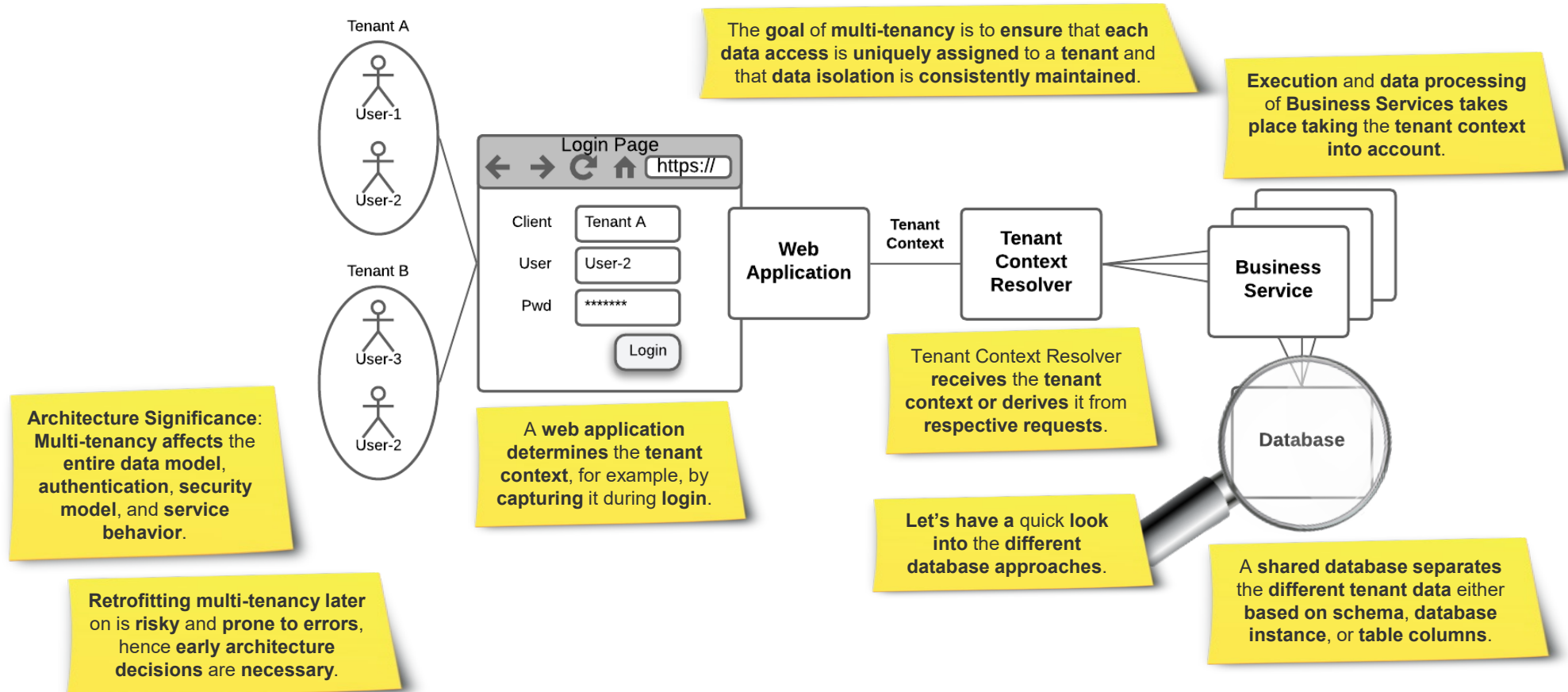


example

Exercise Opening

Requirements Assessment

Requirement 16: The system must be fully multi-client capable.



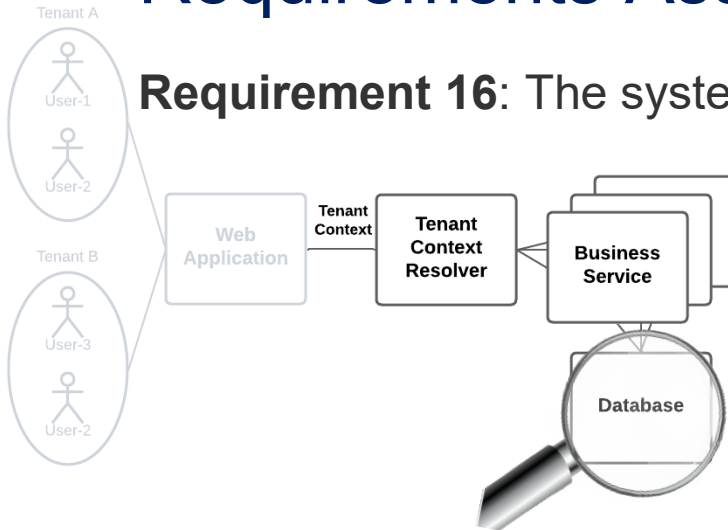


example

Exercise Opening

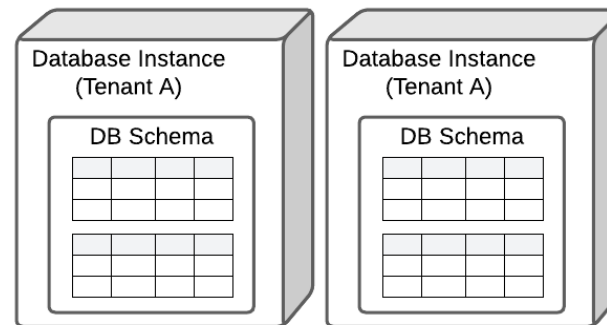
Requirements Assessment

Requirement 16: The system must be fully multi-client capable.



The **goal of multi-tenancy** is to **ensure** that **each data access is uniquely assigned to a tenant** and that **data isolation is consistently maintained**.

Database-based separation (separate database per tenant), where **each tenant** gets its **own physical or logical database instance**.



Each DB has its own users, configurations, indexes, etc.

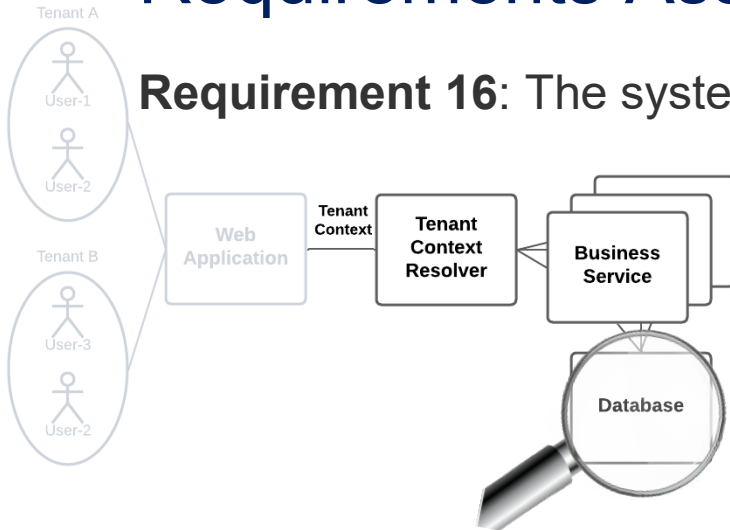


example

Exercise Opening

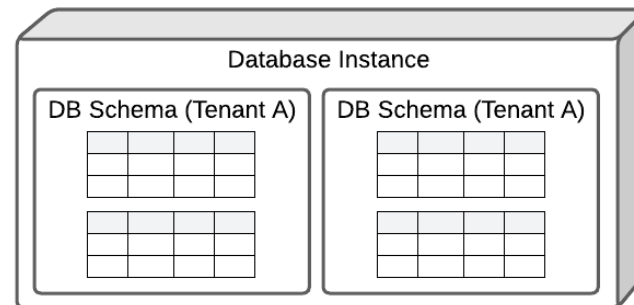
Requirements Assessment

Requirement 16: The system must be fully multi-client capable.



The **goal of multi-tenancy** is to **ensure** that **each data access is uniquely assigned to a tenant** and that **data isolation is consistently maintained**.

Schema-based separation (shared database, separate schema per client), where all clients use the same database instance, but each client has its own database schema.



The **tables** are **identical for each schema** (e.g., schema_a.users, schema_b.users, etc.).

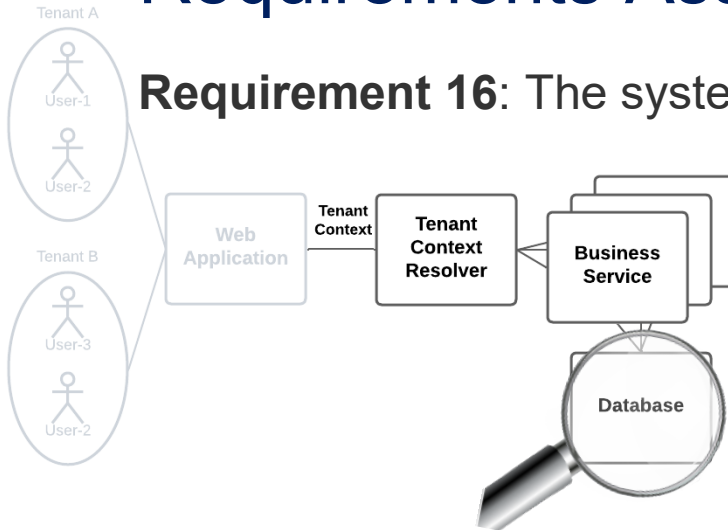


example

Exercise Opening

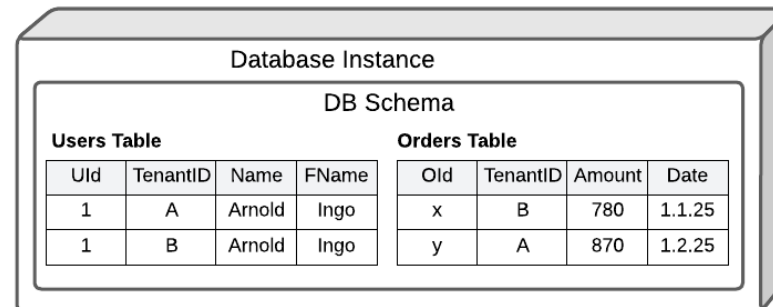
Requirements Assessment

Requirement 16: The system must be fully multi-client capable.



The **goal of multi-tenancy** is to ensure that **each data access is uniquely assigned to a tenant** and that **data isolation is consistently maintained**.

Column-based separation (shared schema, shared tables), in which all tenants share the same tables.



Separation is achieved using a tenant_id column in each table.

For example, users(id, tenant_id, name, email).

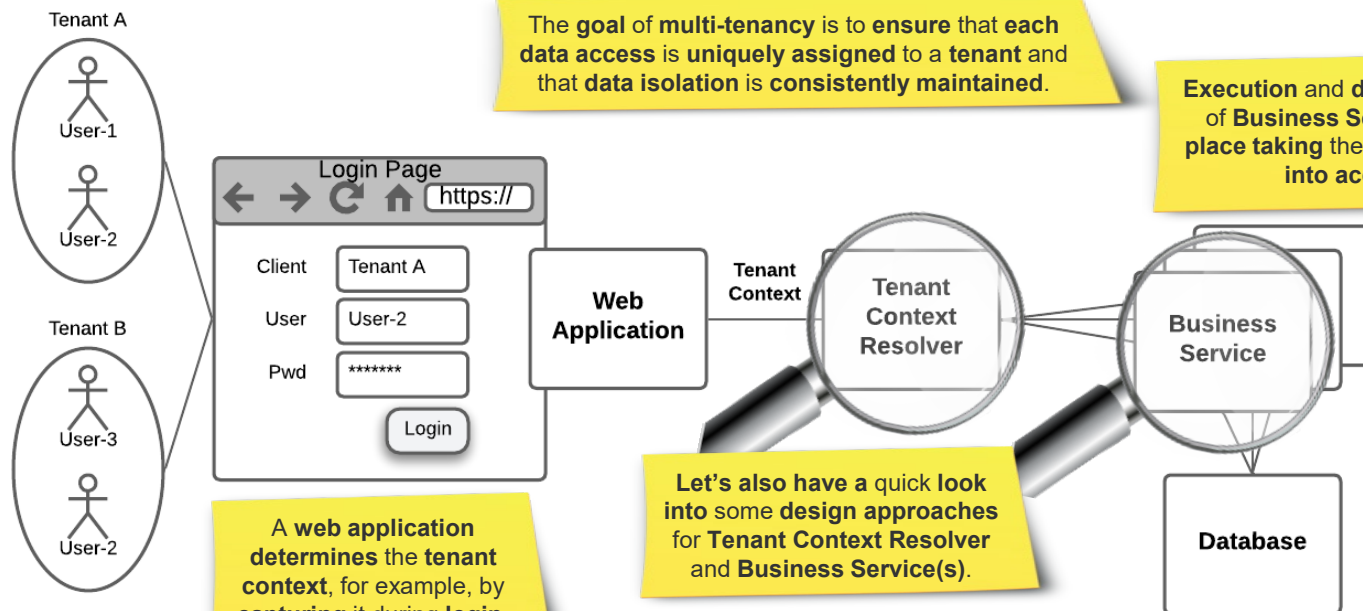


example

Exercise Opening

Requirements Assessment

Requirement 16: The system must be fully multi-client capable.



The **goal of multi-tenancy** is to ensure that **each data access is uniquely assigned to a tenant** and that **data isolation is consistently maintained**.

Execution and data processing of Business Services takes place taking the **tenant context** into account.

Architecture Significance: Multi-tenancy affects the entire data model, authentication, security model, and service behavior.

Retrofitting multi-tenancy later on is **risky** and **prone to errors**, hence **early architecture decisions** are **necessary**.

A web application determines the **tenant context**, for example, by capturing it during login.

Let's also have a quick look into some **design approaches** for **Tenant Context Resolver** and **Business Service(s)**.

A shared database separates the **different tenant data** either based on **schema**, **database instance**, or **table columns**.

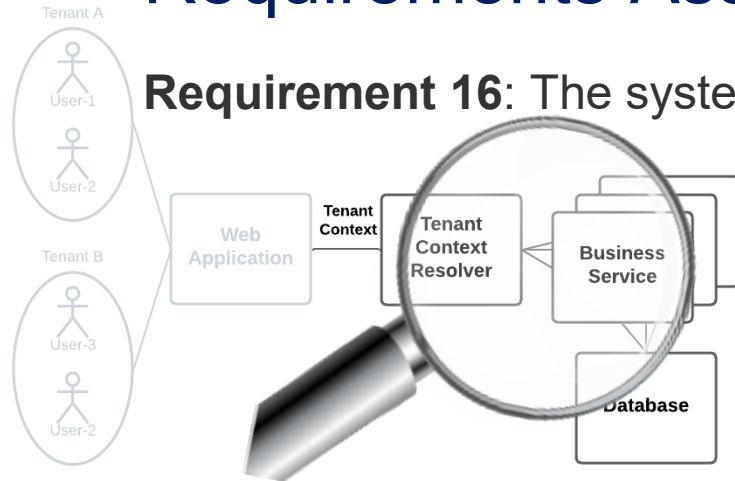


example

Exercise Opening

Requirements Assessment

Requirement 16: The system must be fully multi-client capable.

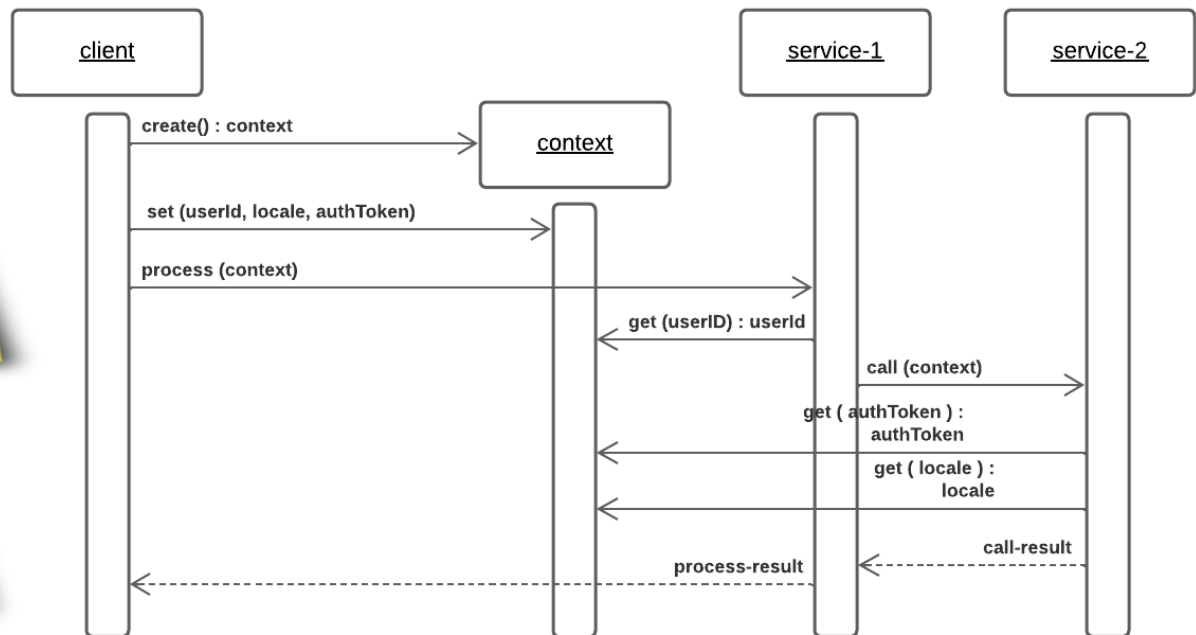


The **goal of multi-tenancy** is to **ensure** that **each data access is uniquely assigned to a tenant** and that **data isolation is consistently maintained**.

The **Context Object pattern** can be used to **encapsulate context (state and behavior)** that is **relevant to a user** or the **request being processed** and **make it available to application components**.

This **decouples application components** from the **complexity of the environment**.

The **design pattern** helps to manage **application context efficiently**.





example

Exercise Opening

Requirements Assessment

Requirement 17: The system must meet real-time requirements (e.g. responses within 50 ms).

Architecture Significance: Yes.

Resulting Architecture Decisions:

Such requirements force specific design decisions, for example regarding memory structure, data storage in RAM, use of asynchronous communication, and avoidance of I/O blocking.

Why difficult to change Architecture Decisions later:

If you design a system without real-time requirements (e.g. with DB-based workflows, heavyweight middleware), converting it to real-time processing is practically a complete rebuild.



example

Exercise Opening

Requirements Assessment

Requirement 18: The system should automatically log out after 15 minutes of inactivity.

Architecture Significance: No.

Resulting Architecture Decisions:

A session timeout rule that can be implemented locally in web server/framework configurations.



example

Exercise Opening

Requirements Assessment

Requirement 19: Input fields should perform validations (e.g. email format).

Architecture Significance: No.

Resulting Architecture Decisions:

Affects the validation logic in the front end or service layer – no significant structural decision.



example

Exercise Opening

Requirements Assessment

Requirement 20: The system must be expandable by third parties (e.g. partner companies) via publicly documented interfaces.

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to the introduction of stable, versioned interfaces, API gateways, developer portals, authentication and authorisation mechanisms for external parties.

Why difficult to change Architecture Decisions later:

A system without clean API concepts is usually closely linked to the internal data model.

Subsequent 'API isolation' requires extensive refactoring and careful encapsulation of internal details.

Requirement 20 leads to **similar considerations regarding architecture-significance** and corresponding **approaches** with regard to **fundamental design decisions**, such as **requirement 4** (→ *All system functions must also be available via a mobile app*).



example

Exercise Opening

Requirements Assessment

Requirement 21: The system must support machine learning (e.g. by training and using models based on usage data).

Architecture Significance: Yes.

Resulting Architecture Decisions:

Requires data pipelines, separation of OLTP and OLAP, data annotations, logging of user interactions, model repository, model versioning.

Why difficult to change Architecture Decisions later:

If the system does not provide for separation of training and production data from the outset, logging, anonymisation and ML infrastructure are difficult to retrofit – especially in a regulatory context.



example

Exercise Opening

Requirements Assessment

Requirement 22: The application should offer a print function for invoices.

Architecture Significance: No.

Resulting Architecture Decisions:

Affects formatting and export, but no fundamental changes to architecture or technology stack.



example

Exercise Opening

Requirements Assessment

Requirement 23: The system should allow users to reset their passwords themselves.

Architecture Significance: No.

Resulting Architecture Decisions:

Standard feature, usually implementable using existing authentication/IAM components; no significant architecture decisions required.



example

Exercise Opening

Requirements Assessment

Requirement 24: The system must support rule-based decision logic that can be adapted by specialists at runtime.

Architecture Significance: Yes.

Resulting Architecture Decisions:

Leads to the separation of business logic and control logic, often also to the introduction of rule engines (e.g. Drools), decision tables, UI components for rules, validation and versioning of rules.

Why difficult to change Architecture Decisions later:

A code-based system can only be converted into a rule-based system with great effort.

Data models, UI and error handling must be partially redesigned.

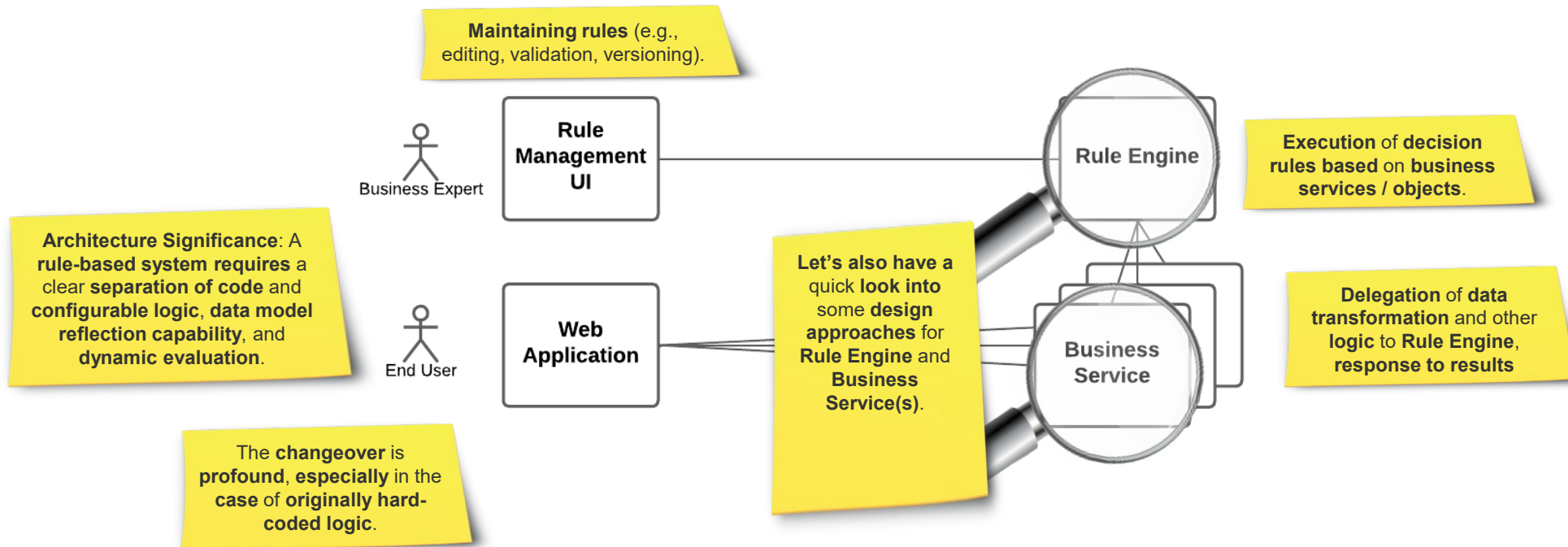


example

Exercise Opening

Requirements Assessment

Requirement 24: The system must support rule-based decision logic that can be adapted by specialists at runtime.



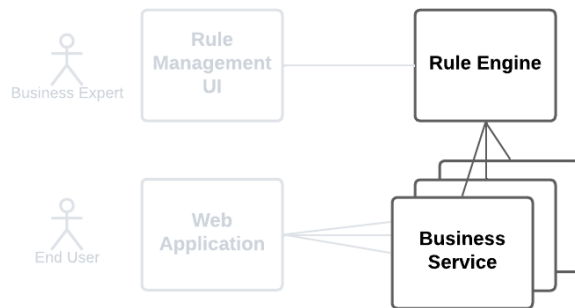


example

Exercise Opening

Requirements Assessment

Requirement 24: The system must support rule-based decision logic that can be adapted by specialists at runtime.

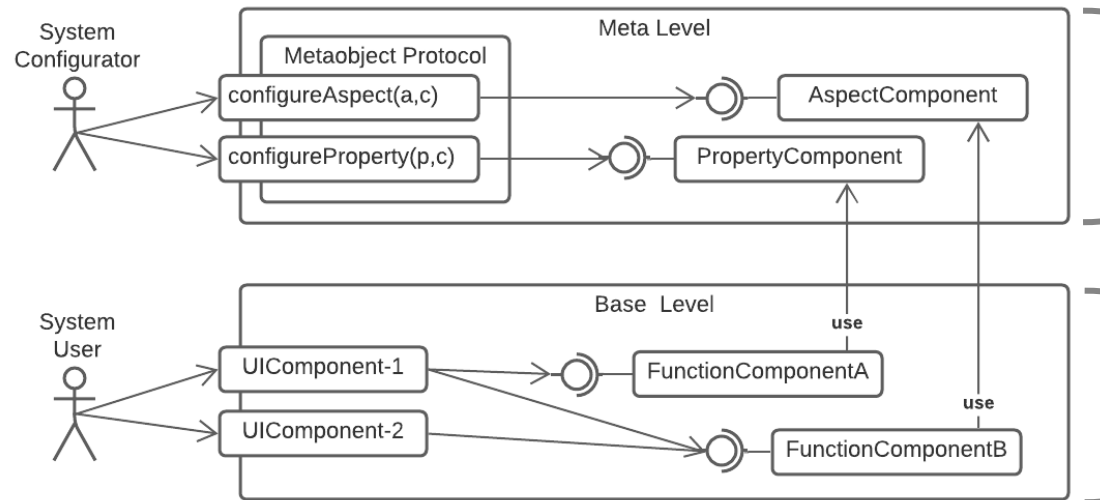


The goal is to **keep rules externally configurable, versionable, and executable, thereby enabling (substantial) changes to be made to the system without having to recompile it.**

An **approach** to achieving high **flexibility** and **adaptability** is a **design based on the Reflection pattern.**

Reflection separates meta-objects from core application logic using a 2-layer architecture:

- A **meta-level** contains the **meta-objects reflecting the modifiable business rules.**
- A **base layer** contains the **business components** that **delegate parts of the business logic to meta-objects** and **consult them for the business logic at runtime.**



Metaobjects are read and interpreted by Base Level components

Functional components realize application logic by reading and interpreting metaobjects on the Meta Level

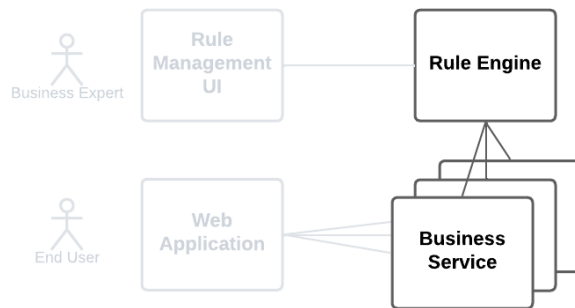


example

Exercise Opening

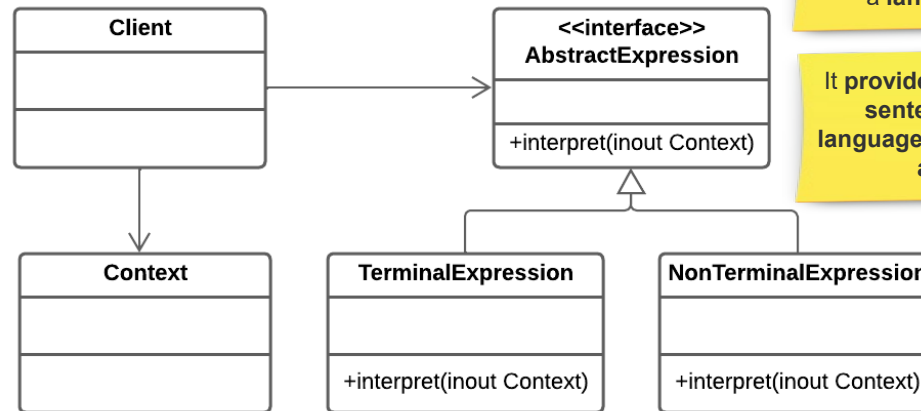
Requirements Assessment

Requirement 24: The system must support rule-based decision logic that can be adapted by specialists at runtime.



The goal is to **keep rules externally configurable, versionable, and executable**, thereby enabling (substantial) **changes** to be made **to the system** without having to **recompile it**.

The **Interpreter pattern** can also be used to **allow flexibility** in **modifying rules** without requiring **recompilation**.



Interpreter is a **behavioural pattern** that **defines a method** for **interpreting** and **evaluating** a **language grammar**.

It provides a **mechanism** for **evaluating sentences** in the **corresponding language** by **representing their grammar** as a **series of classes**.

Perform "parent" functionality, then delegate to each "child" element. Context is a data structure for holding input and output.

Each class represents a **rule or expression** in the **grammar**, and the **pattern** allows these **classes** to be **assembled hierarchically** to **interpret complex expressions**.

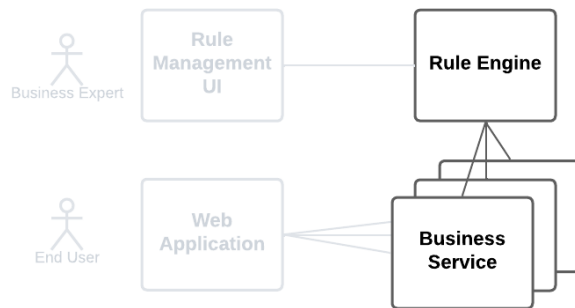


example

Exercise Opening

Requirements Assessment

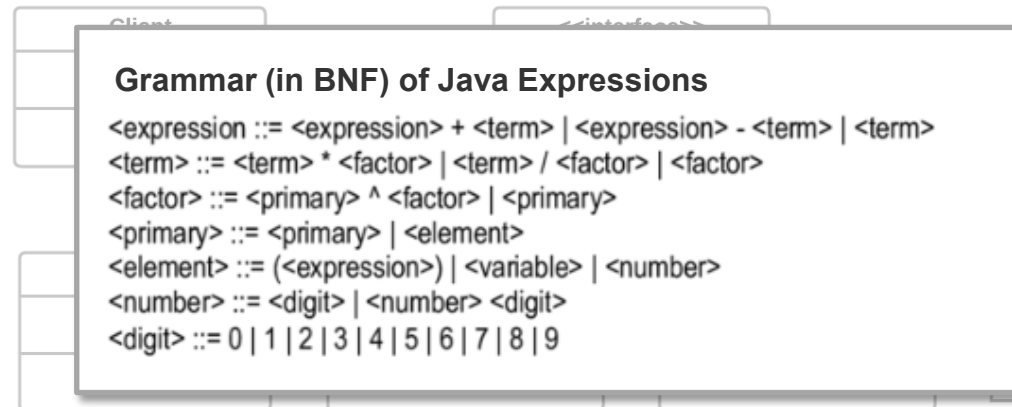
Requirement 24: The system must support rule-based decision logic that can be adapted by specialists at runtime.



The goal is to **keep rules externally configurable, versionable, and executable, thereby enabling (substantial) changes to be made to the system without having to recompile it.**

This is a **grammar example** (in BNF) of a **section of the Java programming language** (i.e., the **grammar of Java expressions**).

Giving an interpreter that is built into a system further access to the business/document object model (DOM) of the surrounding system allows business experts to formulate sophisticated business rules.



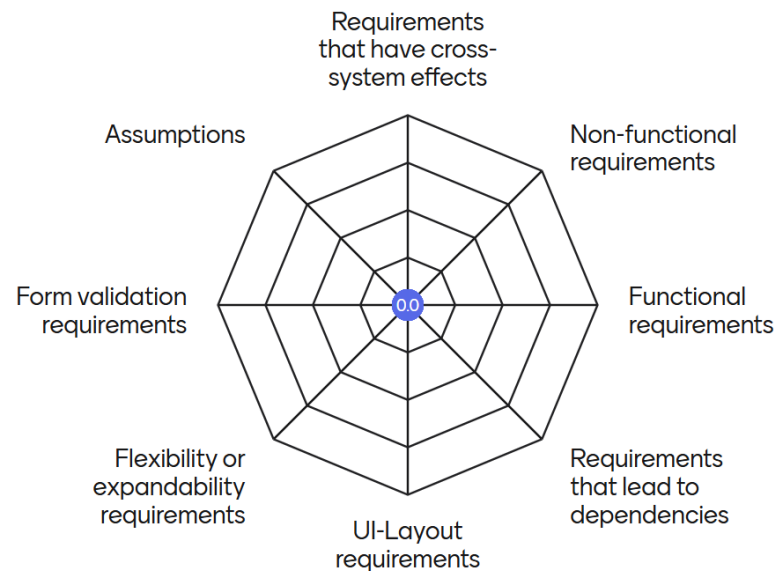
Perform "parent" functionality, then delegate to each "child" element. Context is a data structure for holding input and output.

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Which were **common characteristics** of **requirements** that were **architecture significant**?
2. What is the **extent** to which the following **types of requirements** tend to be **architecture-significant** (answers are provided in the respective Mentimeter)?



Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
Subsequent architectural impact: Requires a configurable UI framework, separation of UI structure and business logic, possibly a metamodel for fields and masks.
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
Subsequent architectural impact: Multilingual database content, language-specific rules (e.g. formatting), language context in APIs – affects persistence and business logic.
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
Subsequent architectural impact: Mobile use requires an API-first approach, offline mode, caching strategies, token-based authentication, push mechanisms – deep intervention in system architecture.
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
Subsequent architectural impact: Increasing role differentiation → attribute-based access control (ABAC), finely granulated rights, integrity protection → affects system model, APIs, security architecture.
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there requirements that do not initially appear to be relevant to the architecture but become so later on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
Subsequent architectural impact: Requirements for data filtering, visibility rules, client protection, performance with large amounts of data → export subsystem with security and configuration logic.
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).

Exercise Opening

Overview

Let us **discuss** the **following questions** in order to **reflect** on the **significance assessments** that we have performed:

1. Are there **requirements** that **do not initially appear** to be **relevant** to the **architecture** but **become so later** on? Let us have a look at a few examples:
 - a. Modifiability in the user interface (Example: “Specialist users should be able to customise UI fields”. Initial assessment: “Only a UI issue, low relevance”).
 - b. Internationalization (Example: “Texts should be displayed in German and English”. Initial assessment: Localisation via UI labels).
 - c. Mobile use (Example: “The app should also work on tablets”. Initial assessment: UI-only issue).
 - d. Role and rights model (Example: “Admin and user rights must be differentiated”. Initial assessment: Implementation in the front end, simple roles).
 - e. Export function as ‘simple CSV’ (Example: “Data should be exportable”. Initial assessment: Pure formatting function).
 - f. Reporting/statistics (Example: “A reporting module would be nice”. Initial assessment: Can be added later).
Subsequent architectural impact: Leads to data warehouse, separate reporting stack (ETL, OLAP), data quality management → strong influence on data architecture.

Wrap-Up

■ Objective of the Exercise

- The **goal** is to **identify which requirements** are **architecturally significant** and to **reason about why they are**.
- **Architecturally significant requirements** (ASRs) are those that:
 - Influence fundamental design decisions.
 - Lead to structures that are difficult to change once implemented.
- **Affect multiple components, layers, or quality attributes** of the system.
- **Students analyse requirements, identify ASRs, and design coarse-grained components** as well as more **detailed architectural approaches** for **selected set of requirements**.

Wrap-Up

■ Understanding Significance

- A requirement is **architecture-significant** if it **strongly shapes** the **system's structure** or **behaviour**.
- **Indicators of significance:**
 - **Impacts non-functional qualities** (e.g., availability, performance, scalability, security).
 - **Requires new architectural mechanisms** or cross-cutting solutions.
 - **Drives irreversible design choices** (platforms, frameworks, integration approaches).
- **Conversely, non-significant requirements typically affect only local elements** (UI, formatting, small validation rules).

Wrap-Up

■ Examples of architecture-significant requirements

1. **High availability (24/7).** Redundant infrastructure (load balancers, failover), distributed deployment, monitoring – difficult to retrofit.
4. **All functions via mobile app.** API-first architecture, authentication tokens, DTOs, adapters, versioning strategies – major design effort.
5. **GDPR compliance.** Requires early data segregation, anonymisation, auditing, access control – foundational to architecture.
8. **Traceability and auditability.** Event sourcing, audit log services, immutable storage, observer pattern – cross-cutting concern.
9. **Integration into SAP.** Imposes technology and integration constraints (e.g. BAPI, IDoc), affects all layers.
12. **Cloud-native deployment.** Microservices, containers, orchestration (Kubernetes), stateless services – foundational redesign.
13. **Configurable by business users.** Requires rule/configuration engines, separation of code and logic, metamodeling.
16. **Multi-tenancy.** Data isolation per tenant (schema, DB, or column-based), tenant context resolver pattern – early decision crucial.
17. **Real-time performance.** In-memory data, async communication, lightweight messaging – cannot easily be added later.

Wrap-Up

- **Examples of architecture-significant requirements** (continued)
 - 20. Extendable by third parties (public APIs).** API gateways, developer portals, authentication and versioning – defines external architecture boundary.
 - 21. Machine learning support.** Data pipelines, model repositories, training/production data separation.
 - 24. Rule-based decision logic (runtime editable).** Rule engine (e.g., Drools), reflection, interpreter pattern – separates logic from code and adds configurability.
- **Patterns and Design Approaches Introduced**
 - **Command Pattern** – encapsulates offline transactions for later synchronization.
 - **DTO Pattern** – optimizes data transfer for APIs/mobile.
 - **Adapter Pattern** – decouples internal models from external APIs.
 - **Event Sourcing Pattern** – ensures traceability and auditability.
 - **Observer Pattern** – automates audit logging and event handling.
 - **Context Object Pattern** – manages multi-tenant context.
 - **Reflection and Interpreter Patterns** – enable dynamic rule evaluation.
 - **Hexagonal (Ports & Adapters) Architecture** – supports cloud-native, decoupled service designs.

Wrap-Up

■ Misleadingly “Non-Significant” Requirements

- **Some requirements may seem trivial at first but later have deep architectural consequences:**
 - **Modifiable UI:** configurable UI frameworks, meta-layer for field definitions.
 - **Internationalization:** multilingual persistence, API adaptations.
 - **Mobile use:** caching, offline mode, authentication tokens.
 - **Roles and rights:** may evolve into attribute-based access control (ABAC).
 - **Export/reporting:** may lead to data warehouse, ETL pipelines, and OLAP systems.

■ Overall Learning Objectives

- **You should be able to ...**
 - **Differentiate between** architecturally **significant and non-significant requirements**.
 - **Explain why** a given requirement leads to **long lasting design decisions**.
 - **Apply heuristics to assess significance** (impact breadth, volatility, conflict potential, stakeholder importance).
 - **Elaborate high-level architecture sketches to investigate design implications**.
 - **Evolve high-level architecture sketches** towards **further detail** in order to **explore design approaches on the next level of detail** (e.g., based on software architecture or design patterns).

Questions

